



Gruppo SkyNet
skynetunigroup@gmail.com

MVP Progettazione Backend

Versione	
Uso	<i>Interno</i>
Redattori	<i>Sara Ristovic</i>
Verifica	<i>Nome Cognome</i>
Approvazione	<i>Nome Cognome</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
1.0	2026/08/14	Creazione e compilazione	Sara Ristovic	Nome Cognome

Indice

1	Perimetro del documento	6
1.1	Obiettivo	6
1.2	Funzionalità degli Agenti supportate nell'MVP	6
1.2.1	Agente Docs	6
1.2.2	Agente Security	7
1.2.3	Agente Changelog	7
1.3	Funzionalità dichiarate ma non implementate nell'MVP	7
1.4	Requisiti fuori dal perimetro del backend	8
1.5	Vincoli tecnologici del backend	9
2	Modello dei dati persistiti	9
3	Identità, Autenticazione e Ruoli	9
3.1	Registrazione	10
3.2	Login e sessione	11
3.3	Logout	11
4	Credenziali dei Servizi Esterni	12
4.1	Configurazione	12
4.2	Validazione	13
4.3	Cifratura (RS.2)	13
4.4	Scope del token ed esecuzione (RS.4)	14
5	Integrazione con GitHub	14
5.1	La facade di sola lettura (invariata dalla PoC)	14
5.2	Cosa cambia nell'MVP	15
5.3	Ciò che resta fuori dalla facade	15
6	Contesto di Analisi: Repository, Riferimento e Ambito	16
6.1	Provenienza delle decisioni	16
6.2	Contratto di creazione del contesto	17
6.3	Sequenza di validazione	18
6.4	Linguaggi supportati	19
6.5	Ambito di analisi	19
6.5.1	I tre tipi di ambito	20
6.5.2	Validazione dell'ambito	20
6.5.3	Limite dimensionale	21
7	Catalogo delle Operazioni e Avvio	21
7.1	Il catalogo delle operazioni	21
7.2	Avvio di un batch	22
7.3	Controlli prima dell'accodamento	22
8	Contratto API	23
8.1	API REST pubbliche	23

8.1.1	Endpoint riutati invariati	23
8.1.2	Endpoint nuovi o modificati	23
8.2	Canale realtime (WebSocket)	25
8.2.1	Eventi di avanzamento e completamento (RF.44, RF.47)	25
8.3	API interne (agente → backend)	26
9	Orchestrazione ed esecuzione	26
9.1	Riuso dalla PoC	26
9.2	Estensioni per l'MVP	27
9.2.1	Catalogo completo delle operazioni (RF.40)	27
9.2.2	Sospensione per input dell'utente (RF.43, RF.98–RF.105)	27
10	Dettaglio degli Agenti	29
10.1	Agente Docs	29
10.1.1	Generazione del README (RF.82)	29
10.1.2	Documentazione inline: generazione e correzione (RF.83, RF.84)	30
10.1.3	Documento API (RF.86)	30
10.2	Agente Security	30
10.2.1	Scansione OWASP (RF.87)	31
10.2.2	Classificazione e localizzazione dei riscontri (RF.88–RF.91)	31
10.2.3	Conformità a POLICY.md (RF.92)	31
10.3	Agente Changelog	31
10.3.1	Changelog tecnico (RF.94–RF.96)	31
10.3.2	Metadati insufficienti (RF.100–RF.102)	31
10.3.3	Changelog di business (RF.103–RF.105)	32
10.3.4	Leggibilità del Changelog di business (RQ.7)	32
11	Gestione degli Errori e Validazione dell'Esecuzione	32
11.1	Riuso dalla PoC	32
11.2	Delta: il dominio di <code>ErrorKind</code>	32
11.3	Dove vive ogni controllo	34
11.4	Limite di utilizzo (RF.66)	34
11.5	Fallimento dell'apertura della Pull Request (RF.72)	35
11.6	Requisiti di qualità sull'esecuzione (RQ.5, RQ.6)	35
11.7	Punti aperti verso il servizio agenti	36
12	Report: Consultazione, Storico ed Esportazione	36
12.1	Riuso dalla PoC	36
12.2	Delta sull'envelope	37
12.3	Delta sui blocchi del corpo (RF.59–RF.62)	37
12.4	Intestazione del report (RF.55–RF.58)	38
12.5	Collegamento alla Pull Request (RF.63)	38
12.6	Modulo di validazione nel report (RF.64)	38
12.7	Storico, consultazione e controllo di accesso (RF.49–RF.53)	39
12.8	Esportazione in PDF (RF.73–RF.74)	40
13	Decisioni Architettureali	41

Elenco delle tabelle

1	Provenienza delle decisioni di progettazione del contesto di analisi.	16
2	Endpoint REST pubblici nuovi o modificati rispetto alla PoC.	23
3	Eventi WebSocket emessi dal backend verso il frontend; i primi quattro riusati identici dalla PoC (§7.3), l'ultimo nuovo per l'MVP (§9.2.2).	26
4	Meccanismo condiviso di sospensione/ripresa della Task Changelog.	29
5	Dominio di ErrorKind nell'MVP. I messaggi mostrati all'utente non sono ridefiniti qui: restano quelli della Progettazione Frontend, Tabella 7.	33
6	Punti da chiudere con il servizio agenti prima della codifica.	43
7	Modifiche all'envelope Report rispetto alla fig. 4 della PoC.	44
8	Modifiche ai blocchi del corpo rispetto alla fig. 3 della PoC.	45

1 Perimetro del documento

1.1 Obiettivo

Il presente documento definisce la progettazione del backend di **Code Guardian** per la release MVP, a partire dai requisiti descritti nell'Analisi dei Requisiti (AdR) e dall'architettura validata nel Proof of Concept (PoC). Le scelte già validate nella PoC vengono riutilizzate come base implementativa; le funzionalità non coperte in quella fase vengono qui progettate ex novo. Tutte le scelte rispettano i vincoli definiti nei documenti di Progettazione Infrastruttura AWS e Progettazione Frontend, ai quali questo documento si allinea invece di ridefinire decisioni già prese altrove.

1.2 Funzionalità degli Agenti supportate nell'MVP

Questa sezione elenca *quali* funzionalità di ciascun agente il backend deve orchestrare nell'MVP, con riferimento ai requisiti obbligatori corrispondenti. Il *come* verrà dettagliato nelle sezioni successive, una volta completata la progettazione dei componenti che le espongono.

1.2.1 Agente Docs

Rispetto alla PoC, che copriva solo la documentazione inline, l'MVP promuove a requisito obbligatorio l'intera famiglia di operazioni dell'Agente Docs:

- **RF.82** — Generazione e proposta di aggiornamento del `README.md` di progetto, tramite apertura automatica di una Pull Request.
- **RF.83** — Analisi del codice sorgente e proposta di aggiornamento della documentazione Inline (JSDoc), tramite apertura automatica di una Pull Request.
- **RF.84** — Correzione della documentazione Inline esistente quando non più coerente con il codice (non solo rilevamento di documentazione mancante, come nella PoC).
- **RF.86** — Estrazione delle informazioni relative agli Endpoint e generazione del documento `API.md`, tramite apertura automatica di una Pull Request.
- **RF.61** — Segnalazione strutturata (categoria e gravità) quando il codice è troppo complesso per essere documentato in modo affidabile; applicabile solo alle operazioni di documentazione Inline e API, non al README.
- **RF.63** — Collegamento alla Pull Request generata, esposto nel report.

L'apertura delle tre Pull Request avviene sempre secondo il meccanismo descritto in ADR-4: mai da parte dell'agente, sempre e solo dal backend, in modo automatico al termine dell'esecuzione.

1.2.2 Agente Security

- **RF.87** — Scansione del codice sorgente nell'ambito selezionato per individuare pattern riconducibili alle vulnerabilità dell'OWASP Top 10.
- **RF.88–RF.89** — Classificazione di ogni vulnerabilità per categoria OWASP e gravità, con associazione al file e, se disponibile, alla porzione di codice coinvolta.
- **RF.90–RF.91** — Generazione di un suggerimento di remediation: uno Snippet di codice correttivo quando il problema è sufficientemente localizzato, altrimenti una descrizione testuale.
- **RF.92–RF.93** — Verifica di conformità rispetto alle regole di sviluppo sicuro definite in `POLICY.md`, limitata alle sole direttive pertinenti alla sicurezza del software. A differenza della PoC, dove questa variante era opzionale, nell'MVP è parte integrante e obbligatoria dell'Agente Security.

Non implementato nell'MVP: copertura esaustiva dell'intera OWASP Top 10; applicazione automatica dei rimedi proposti; apertura di Pull Request per le vulnerabilità rilevate. L'Agente Security resta *flag-only*: individua e documenta, ma è il Security Auditor a intervenire manualmente sul codice, al di fuori di Code Guardian.

1.2.3 Agente Changelog

- **RF.94–RF.95** — Interrogazione delle GitHub Issues (che, per decisione di team, coincidono con il sistema di Task Management) per recuperare e filtrare le User Stories e i Task completati dello Sprint richiesto, estraendone i metadati rilevanti.
- **RF.96** — Generazione di un Changelog tecnico strutturato, orientato allo sviluppo.
- **RF.98–RF.99** — Richiesta dello Sprint ID all'utente durante l'esecuzione, con possibilità di annullamento esplicito prima che l'agente venga effettivamente avviato.
- **RF.100–RF.102** — Rilevamento dei Task con metadati insufficienti, con scelta esplicita dell'utente tra procedere escludendoli o annullare l'intera operazione.
- **RF.103–RF.105** — Generazione del Changelog di business come fase interna della stessa Task (non una nuova operazione separata), attivata previa conferma dell'utente sul Changelog tecnico appena generato, con possibilità di annullamento anche in questa fase.
- **RQ.7** — Verifica dell'indice di leggibilità Flesch Reading Ease (> 50) su ogni Changelog di business generato.

1.3 Funzionalità dichiarate ma non implementate nell'MVP

A differenza di §1.4, che elenca requisiti non di competenza del backend, questa raccoglie funzionalità che *sarebbero* di competenza del backend ma che l'MVP, per decisione esplicita, non costruisce. Le esclusioni specifiche di ciascun agente restano nelle rispettive sottosezioni sopra; qui vanno le esclusioni trasversali.

- **Pull Request come riferimento di base (RF.23, con riflesso su RF.16)** — Nell'Analisi dei Requisiti, **RF.18** (riferimenti basati su Pull Request) è desiderabile, non obbligatorio; di conseguenza **RF.23**, che ne verificherebbe l'esistenza, non è implementato. Il riferimento di base nell'MVP è sempre un branch, eventualmente ancorato a un commit (§6): non esiste alcun selettore del tipo di riferimento, e la seconda metà di **RF.16** — “selezionare la tipologia di riferimento di base” — risulta quindi soddisfatta con un'unica tipologia possibile.

1.4 Requisiti fuori dal perimetro del backend

I seguenti requisiti, pur presenti nell'Analisi dei Requisiti non richiedono alcuna decisione progettuale da parte del backend:

- **RV.9** — Lo sviluppo del frontend in TypeScript/React è una scelta di stack che riguarda esclusivamente il documento di Progettazione Frontend.
- **RV.17** — Il Manuale Utente è un deliverable di prodotto trasversale, non specifico del backend.
- **RV.18** — Il documento di Specifiche Tecniche è redatto dall'intero team; questo stesso documento ne costituisce la parte relativa al backend, senza necessitare di una voce dedicata separata.
- **RF.93** — quali direttive di `POLICY.md` siano pertinenti alla sicurezza è un'interpretazione del contenuto del file, guidata dal template di scansione dell'Agente Security. Il backend non decide né verifica quali regole siano incluse.
- **RF.95** — l'estrazione dei metadati dai Task validati avviene interamente nel loader dell'Agente Changelog, prima dell'invocazione del modello. Il backend non vede mai i singoli campi delle Issue, solo il Report finale dell'agente.
- **RQ.4** — l'isolamento dei prompt dai modelli linguistici vive interamente nel `PromptRepository` del servizio agenti (Python): il backend non contiene né referencia alcun prompt.
- **RV.2** — l'integrazione di modelli pre-addestrati esclusivamente via API è una proprietà dell'astrazione `LLMProvider` del servizio agenti; il backend non chiama mai direttamente un LLM.
- **RV.11** — lo sviluppo del sottosistema ad agenti in Python/LangGraph è una scelta implementativa del servizio agenti, non del backend, che si limita al contratto tra i due servizi (§5).
- **RV.13** — l'integrazione con AWS Bedrock avviene interamente tramite `LLMProvider` nel servizio agenti: Progettazione AWS §6.2 assegna i permessi `bedrock:InvokeModel` al Task Role *degli agents*, mai a quello del backend.
- **RS.5** (*desiderabile*) — la garanzia sul trattamento dei dati inviati al modello dipende dal canale configurato in `LLMProvider` (Bedrock in produzione, i cui termini sono dichiarati da AWS); nessuna decisione backend ne consegue.

Restano invece di competenza backend, seppure con un contributo parziale o di processo più che architetturale, RV.5, RQ.10 (di cui il backend garantisce solo la propria quota del tempo di risposta complessivo), RQ.11 (desiderabile; nessuna progettazione dedicata, il contributo indiretto sono gli errori per campo di RF.6 e il catalogo chiuso di `error.code`, §11), RQ.1–RQ.3, RV.16 (riuso invariato di Swagger per gli endpoint pubblici, §8.1; resta aperta, come decisione di team, se il perimetro interno HMAC richieda una specifica separata), RV.19–RV.20 (obblighi di qualità, testing e documentazione tecnica che il backend soddisfa nella propria implementazione, senza richiedere qui una progettazione dedicata) e RV.3 (nessun disegno proprio: discende da RV.4 e RV.7, verificato eseguendo gli agenti sui repository del Proponente in fase di validazione). RV.21 è invece soddisfatto da §2 (Modello dei dati persistiti, in redazione).

1.5 Vincoli tecnologici del backend

Il backend è sviluppato in TypeScript con il framework NestJS (**RV.10**), invariato dalla PoC fin dalla prima stesura. La persistenza usa MongoDB Atlas (**RV.12**), anch'esso già in uso nella PoC tramite Mongoose: l'MVP estende le collezioni esistenti (§2) invece di cambiare database. Nessuno dei due è un vincolo che richieda una decisione progettuale: sono scelte di stack già esercitate end-to-end.

L'hosting su AWS (**RV.15**) è invece un vincolo che la PoC dichiara esplicitamente fuori dal proprio perimetro (§2.2: “l'infrastruttura AWS ...sostituita da esecuzione locale containerizzata”), ma che non richiede una riscrittura: Progettazione AWS applica al backend lo stesso principio di migrazione di ogni altro servizio, “stessa unità di deploy, cambia solo la configurazione”, dimensionandolo come servizio ECS Fargate distinto dagli agenti (§4.1, 0.5 vCPU, 1 GB, **Desired Count**: 1). Ciò che resta da vincolare è solo il comportamento del container perché sia un buon cittadino di quella infrastruttura: leggere configurazione e segreti da variabili d'ambiente a runtime, non da valori cablati; non dipendere da alcuno stato sul filesystem locale che non sopravviverebbe a un riavvio del container; ed esporre l'endpoint di health check che l'infrastruttura monitora (GET `/auth/health`). L'MVP mantiene il backend a un **Desired Count** fisso pari a 1 — una semplificazione infrastrutturale deliberata, non un requisito di questo documento — quindi la piena statelessness per lo scaling orizzontale resta fuori dal perimetro di questa release.

2 Modello dei dati persistiti

[Da completare — schemi MongoDB estesi rispetto al modello della PoC (§4.3): `User`, `ServiceCredential`, `AnalysisContext`, `Task`, `Report`, `AccessLog`,...]

3 Identità, Autenticazione e Ruoli

La PoC espone gli endpoint POST `/auth/register` e POST `/auth/login` come stub (§7.2): rispondono, ma non creano né verificano alcun utente. Tutto il contenuto di questa sezio-

ne è quindi progettazione ex novo; della PoC si riusa solo la forma del contratto (percorso, verbo, prefisso `/api/v1`).

3.1 Registrazione

`POST /auth/register` (pubblico, nessun JWT) accetta `RegisterDto` e restituisce 201 con `UserProfileDto`, **senza** token: l'utente si autentica separatamente tramite `POST /auth/login`. Nessun auto-login è previsto: lo conferma il flusso della Progettazione Frontend (§3.2), dove solo `/login` definisce uno stato di successo esplicito (§4.2–4.3).

Gli schemi, sul modello di quelli già definiti dalla PoC (§7.4), fissano non solo i campi ma anche le regole di validazione da applicare, in modo che l'implementazione non lasci spazio a controlli manuali divergenti:

```
// --- Body di POST /auth/register ---
interface RegisterDto {
    firstName: string          // 1-40 caratteri
    lastName: string           // 1-40 caratteri
    email: string               // formato email valido
    password: string            // minimo 8 caratteri, almeno una lettera
                                // e una cifra
    role: UserRole
}

// --- Risposta di POST /auth/register e di GET /auth/me ---
interface UserProfileDto {
    id: string
    firstName: string
    lastName: string
    email: string
    role: UserRole
}
```

Regole applicate:

- **Unicità dell'email (RF.5)** — indice `unique` su `User.email` come garanzia autoritativa (protegge da due registrazioni concorrenti), con un controllo applicativo preliminare per un errore 409 leggibile. Email normalizzata (`trim + minuscolo`) prima di entrambi.
- **Validazione (RF.6)** — email in formato valido; password di almeno 8 caratteri, con almeno una lettera e una cifra. Errori 400 con dettaglio per campo.
- **Ruolo (RF.4)** — validato contro l'enum chiuso già definito nel modello della PoC (`DEVELOPER`, `SECURITY_AUDITOR`, `PROJECT_MANAGER`). Nell'MVP non è previsto nessun endpoint di cambio ruolo, poiché l'utente sceglie il proprio ruolo esclusivamente in fase di registrazione.

- **Password (RF.3, RS.1)** — l’hash Argon2id, calcolato immediatamente alla ricezione, è ciò che viene persistito in `User.passwordHash`; il salt per-utente è incluso in quella stessa stringa, nessun campo separato. Il testo in chiaro non viene *mai* persistito né loggato: vive solo per la durata della chiamata di hashing (in registrazione) o di verifica (in login), poi viene scartato.

3.2 Login e sessione

`POST /auth/login` accetta `LoginDto`, verifica con Argon2id e, in caso di successo, restituisce `AuthTokenDto`: un JWT (HS256, `JWT_SECRET` da AWS Secrets Manager) con i claim `sub`, `role`, `iat`, `exp`. Scadenza 8 ore, **nessun refresh token**: la Progettazione Frontend scarta già la sessione a ogni ricaricamento della pagina (ADR-FE-6), quindi un rinnovo silenzioso non servirebbe ad alcuno scenario reale. Lo stesso JWT autentica sia le chiamate REST sia l’handshake WebSocket (§7.3) — un’unica credenziale, non due meccanismi distinti.

```
// --- Body di POST /auth/login ---
interface LoginDto {
    email: string
    password: string           // nessun vincolo di formato qui: si
                              // verifica contro l’hash, non si
                              // richiede una forma
}

// --- Risposta di POST /auth/login ---
interface AuthTokenDto {
    accessToken: string
}
```

Il ruolo viaggia nel token (**RF.7**): un guard lo verifica su ogni endpoint protetto, `GET /operations` restituisce solo le operazioni permesse al ruolo del chiamante, e `POST /tasks` lo ricontrolla comunque, restituendo 403 se l’operazione non è permessa. La lista filtrata lato frontend resta una comodità di interfaccia, non il confine di sicurezza, e la mappatura ruolo→operazione non viene ridefinita qui: quella della Progettazione Frontend resta l’unica fonte di verità.

Le credenziali non valide restituiscono un unico 401 generico (**RF.9**), identico per email inesistente o password errata, per non consentire l’enumerazione degli account. Rate limiting e blocco dell’account dopo tentativi ripetuti **non** sono implementati nell’MVP: nessun requisito obbligatorio li impone, e nessun WAF è attivo davanti al backend. È un rischio accettato consapevolmente, non una svista.

3.3 Logout

`POST /auth/logout` è una rotta pubblica che richiede comunque un JWT valido, e non fa nulla lato server oltre a rispondere 204: essendo il JWT stateless, non esiste alcuna sessione da invalidare. Esiste solo perché il comando “Esci” dell’intestazione applicativa

(Progettazione Frontend, §4.10) prevede uno stato esplicito di *fallimento di rete*: serve quindi una vera chiamata di rete a cui quello stato possa applicarsi, non solo uno scarto locale del token.

4 Credenziali dei Servizi Esterni

La PoC espone già i quattro endpoint necessari (POST /credentials, GET /credentials, POST /credentials/:id/validate, DELETE /credentials/:id) e il modello dati (ServiceCredentialDto §4.3): ciò che manca è il ciclo di vita completo attorno a essi, escluso esplicitamente dal perimetro della PoC (§2.2).

4.1 Configurazione

POST /credentials accetta CreateCredentialDto e restituisce ServiceCredentialDto:

```
// --- Body di POST /credentials ---
interface CreateCredentialDto {
    provider: string          // "GITHUB" nell'MVP
    token: string             // Personal Access Token in chiaro,
                             // mai persistito così com'è
}

// --- Risposta di POST /credentials, elemento di GET /credentials ---
interface ServiceCredentialDto {
    id: string
    provider: string
    connectedAt: string       // data dell'ultima validazione riuscita
}
```

Nell'MVP **provider** resta una stringa libera, non un enum chiuso, coerentemente con la schermata a lista generica di Frontend §4.4 e con ADR-FE-4: un futuro provider aggiuntivo significa una nuova voce nella lista, non una migrazione di schema. Un solo servizio è configurabile, GitHub, il cui token copre sia l'accesso al codice (**RF.11**) sia l'accesso al sistema di Task Management (**RF.12**), per la decisione di team già introdotta in §1.2.3. Restare una stringa libera non significa restare non validata: il valore è verificato contro una whitelist a runtime dei provider attualmente supportati (oggi, solo "GITHUB"), respinto con 400 altrimenti. L'assenza di questa opzione nell'interfaccia del Frontend non è di per sé un confine di sicurezza, lo stesso principio già applicato in §3.

Vale l'invariante un **ServiceCredential** per (userId, provider), imposto da un indice composto **unique**: protegge da un doppio click, una richiesta ripetuta o una POST duplicata che creerebbero due credenziali per lo stesso utente e provider, lasciando ambiguo quale delle due la facade debba usare. Riconnettere un provider già configurato sostituisce la credenziale esistente invece di crearne una seconda. DELETE revoca solo localmente: rimuove il ciphertext persistito, ma non revoca il token su GitHub, azione che resta all'utente.

4.2 Validazione

La PoC disegna un solo endpoint di validazione, `POST /credentials/:id/validate`, ma questo agisce su una credenziale già salvata — l'opposto di quanto richiede **RF.13**, che impone di interrompere il salvataggio in caso di fallimento. Frontend §3.2 conferma la sequenza corretta: validazione sintattica e chiamata di test prima di persistere. Servono quindi due momenti distinti, non uno solo:

1. **Al salvataggio** (`POST /credentials`, **RF.13–RF.14**) — prima il formato (controllo di forma, non un pattern rigido, dato che il formato dei token GitHub è cambiato nel tempo), poi una chiamata di test a GitHub che verifica raggiungibilità e scope. Solo se entrambe passano, la credenziale viene cifrata e persistita; altrimenti non viene scritto nulla.
2. **A richiesta** (`POST /credentials/:id/validate`) — ri-valida una credenziale già salvata, dietro il pulsante “Verifica di nuovo” di Frontend §4.4. Serve perché un token può invalidarsi su GitHub in qualunque momento, senza che Code Guardian ne sia notificato.

4.3 Cifratura (RS.2)

Lo schema della PoC (`ciphertext`, `iv`, `salt`, `authTag`) resta invariato, ma la PoC fissa solo la *forma* dei dati e la dicitura generica “cifratura AES” — non l'algoritmo esatto né la funzione di derivazione della chiave. Non essendo specificati altrove, sono una decisione di questo documento.

Flusso, dal primo invio del token alla sua persistenza.

1. L'utente invia il token in chiaro dal browser a `POST /credentials`, su HTTPS (TLS terminato a CloudFront, Progettazione AWS §8.2).
2. Il backend è l'unico componente che vede mai il token in chiaro: non raggiunge Redis, non raggiunge il servizio agenti, non viene mai loggato.
3. Prima il controllo di formato (**RF.14**), poi la chiamata di test a GitHub (**RF.13**, §4.2): solo se entrambi passano si procede oltre.
4. **Cifratura.** Il backend deriva, tramite HKDF, una chiave specifica per questo record a partire da `CREDENTIAL MASTER KEY` e da un `salt` generato casualmente — invece di cifrare ogni credenziale direttamente con la chiave master, così che una singola chiave derivata compromessa non esponga né la master key né le altre credenziali. Con quella chiave derivata e un `iv` casuale, il token viene cifrato con **AES-256-GCM**, producendo `ciphertext` e `authTag`: la modalità GCM è autenticata, quindi un `ciphertext` alterato viene rilevato in fase di decifratura invece di restituire silenziosamente un valore errato.
5. I quattro campi vengono persistiti nel documento `ServiceCredential` su MongoDB Atlas; il token in chiaro esce di scope e non viene scritto altrove.

6. Alla lettura, quando un agente deve leggere da GitHub tramite la facade (§5), il backend ripete la derivazione con lo stesso `salt` e decifra al volo: nessuna copia decifrata viene mai conservata, coerentemente con ADR-1.

Perché HKDF e non Argon2id/scrypt (**RS.1**): quelle funzioni sono deliberatamente lente, pensate per derivare chiavi da segreti a bassa entropia come una password umana. La chiave master è già ad alta entropia, generata da AWS Secrets Manager: una funzione di derivazione lenta non aggiungerebbe sicurezza, solo latenza.

Ciò che l'MVP rende inoltre esplicito è dove vive la chiave master: `CREDENTIAL_MASTER_KEY` è un segreto in AWS Secrets Manager, cifrato a riposo dalla CMK KMS condivisa (Progettazione AWS §9.1–9.2) — un sistema diverso da MongoDB Atlas, sotto un provider diverso, così che leggere il solo database non restituisca nulla di utilizzabile.

4.4 Scope del token ed esecuzione (**RS.4**)

Un solo token GitHub con scope `repo`, uniforme per tutti e tre i ruoli operativi anziché segmentato per ruolo: la scelta è di Progettazione Infrastruttura AWS (Tabella 20, riga “Token GitHub”), non di questo documento, e sacrifica consapevolmente il principio di privilegio minimo in cambio di una sola credenziale e una sola istruzione per l'utente. Non indebolisce **RS.3**: il confine di sicurezza non è mai stato lo scope del token, ma la whitelist della facade e l'isolamento di rete degli agenti (§5, ADR-AWS-10) — un agente Security non scrive mai, non perché il suo token non lo permetta, ma perché il percorso di codice che usa non lo consente.

Lo scope viene verificato con una vera chiamata a GitHub una sola volta, al salvataggio (**RF.13**): ripeterla a ogni avvio di operazione aggiungerebbe una chiamata esterna e latenza prima di ogni singola Task, contro il budget di **RQ.10**, senza un beneficio reale. Il controllo prima dell'esecuzione (**RF.42**) resta quindi solo un controllo di esistenza — la credenziale è configurata? — non una nuova verifica dello scope. Se un token già salvato risulta invalido, lo si scopre durante l'esecuzione, nel momento in cui la facade tenta davvero la chiamata a GitHub: la Task fallisce con il codice `CREDENTIAL_INVALID`, che il frontend riceve via WebSocket e usa per mostrare il proprio banner persistente di riconfigurazione (Frontend §2.3). Un'irraggiungibilità di rete verso GitHub resta un caso distinto, riportato come fallimento generico di orchestrazione e non come errore di credenziale (Progettazione AWS, Tabella 20, riga “NAT Gateway”).

5 Integrazione con GitHub

Questa sezione è quasi interamente riuso: la PoC (§6) progetta già l'intera integrazione con GitHub, e l'MVP la estende in due punti soltanto, senza toccarne il meccanismo.

5.1 La facade di sola lettura (invariata dalla PoC)

L'agente non possiede il token né conosce l'URL di GitHub: quando deve leggere qualcosa invoca un *tool*, che è in realtà una chiamata a un endpoint interno del backend. È il backend a decifrare il token, chiamare GitHub in sola lettura, registrare l'accesso e

restituire il contenuto. Tre soli tool esistono, tutti di lettura: `read_tree`, `read_file`, `read_issues`. Questa inversione è **ADR-1** ed è ciò che tiene il token confinato in un unico servizio.

Ogni chiamata attraversa quattro controlli, implementati una volta sola nel client GitHub (PoC §6.2): la `ReadOnlyEndpointWhitelist`, che rende **RS.3** garantito per costruzione e non affidato all'attenzione di chi scrive il codice; la cache Redis con chiave `repo@sha:path`, sicura perché il contesto è ancorato a uno SHA e il contenuto di un file a quel commit è immutabile; il *rate limit guard*, che rallenta in prossimità dell'esaurimento della quota invece di fallire; e l'`AccessLog`, che rende **RS.3** dimostrabile invece che dichiarato. Gli endpoint interni restano autenticati via HMAC e fuori dallo Swagger pubblico (PoC §6.3).

Nel complesso questo soddisfa **RV.14** e il versante di lettura di **RS.3**.

5.2 Cosa cambia nell'MVP

Una sola estensione, additiva: la verifica di appartenenza di un commit a un branch (**RF.22**, §6.3) richiede una lettura che la PoC non usava. Non è un tool esposto agli agenti: è un metodo del `GitHubClient` interno, chiamato direttamente dal backend durante `POST /contexts`, nello stesso modo in cui `listRefs` e `resolveRefToSha` sono già usati oggi.

```
compareCommits(userId, owner, repo, base, head): CompareResult
```

```
interface CompareResult {
  status: 'ahead' | 'behind' | 'identical' | 'diverged'
}
```

Chiamato come `compareCommits(userId, owner, repo, base=commitSha, head=branchHeadSha)`, il commit appartiene al branch se `status` è `'ahead'` (il branch è avanti rispetto al commit, cioè il commit ne è un antenato) o `'identical'`; `'behind'` o `'diverged'` significano che non vi appartiene.

Poiché **RS.3** vincola solo gli agenti autonomi, non le chiamate interne del backend, e la `ReadOnlyEndpointWhitelist` esiste per governare le sole richieste che gli agenti indirizzano attraverso la facade, il percorso GitHub sottostante (`GET /repos/owner/repo/compare/base...head`) non va aggiunto lì: è una chiamata diretta del backend, fuori da quel perimetro, sullo stesso piano dell'apertura delle Pull Request.

5.3 Ciò che resta fuori dalla facade

L'apertura delle Pull Request (**RF.82**, **RF.83**, **RF.86**) è un percorso di scrittura del solo backend, deliberatamente esterno a questa facade di sola lettura: né i tre tool né la whitelist vi partecipano. Il rationale completo è in **ADR-4**.

6 Contesto di Analisi: Repository, Riferimento e Ambito

Il contesto è ciò che `POST /contexts` produce e che ogni operazione successiva riusa. La PoC ne progetta già la persistenza e il modello; l'MVP ne cambia il contratto d'ingresso, perché i requisiti sul riferimento di base chiedono più di quanto il campo unico della PoC potesse esprimere.

6.1 Provenienza delle decisioni

Tabella 1: Provenienza delle decisioni di progettazione del contesto di analisi.

Req.	Provenienza	Sintesi
RF.15	Riuso	<code>POST /contexts</code> e <code>AnalysisContext</code> persistito, invariati.
RF.16	Modificato	Il backend riceve l'URL, non <code>repoOwner/repoName</code> già separati.
RF.17	Modificato	Il campo unico <code>ref</code> diventa <code>branch + commitSha?</code> .
RF.19	Nuovo	Validazione sintattica dell'URL lato server.
RF.20	Nuovo	Raggiungibilità e accesso come passo esplicito della creazione.
RF.21	Modificato	<code>listRefs</code> riusato per verificare, non solo per autocompletare.
RF.22	Nuovo	Verifica di appartenenza del commit; nuova voce in whitelist.
RF.24	Riuso	<code>detectLanguages</code> e <code>assertSupportedLanguages</code> invariati.
RV.3	Riuso	Nessun design proprio; discende da RV.4 e RV.7.
RV.4	Riuso	<code>isPrivate</code> e token dell'utente.
RV.7	Riuso	Insieme dei linguaggi supportati.
RF.25	Modificato	L'endpoint di navigazione dell'albero riceve l'URL, non <code>owner/repo</code> separati, e restituisce anche <code>detectedLanguages</code> .
RF.26	Riuso	<code>FULL REPOSITORY</code> invariato.
RF.27	Modificato	<code>FILES</code> invariato; i percorsi sono ora normalizzati prima della validazione.
RF.28	Modificato	<code>DIRECTORIES</code> è ora un prefisso, espanso ricorsivamente alla lettura.
RF.29	Nuovo	Ambito vuoto per <code>FILES/DIRECTORIES</code> ora respinto con 400.
RF.30	Nuovo	Ogni percorso selezionato è verificato contro l'albero a <code>resolvedSha</code> .
RF.31	Riuso	Guardiano dimensionale lato agente invariato; nessuna aggiunta lato backend.

6.2 Contratto di creazione del contesto

```
// --- Body di POST /contexts ---
interface CreateContextDto {
    repoUrl: string          // MODIFICATO: sostituisce repoOwner + repoName
    branch: string           // MODIFICATO: sostituisce il campo unico ref
    commitSha?: string       // MODIFICATO: opzionale, era anch'esso in ref
    scopeType: 'FULL_REPOSITORY' | 'FILES' | 'DIRECTORIES'
    paths?: string[]        // richiesto se scopeType != FULL_REPOSITORY
}

// --- Risposta di POST /contexts ---
interface AnalysisContextDto {
    id: string
    repoOwner: string        // derivati da repoUrl dal backend
    repoName: string
    isPrivate: boolean
    branch: string
    resolvedSha: string      // commit a cui il contesto resta ancorato
    scopeType: string
    detectedLanguages: string[]
    estimatedFileCount: number
}
```

Rispetto alla PoC (§7.4) cambiano quattro cose, tutte nel corpo della richiesta:

- **repoUrl** al posto di **repoOwner** e **repoName** (**RF.16**). La PoC assumeva che la separazione fosse già avvenuta a monte, ma l'utente digita un URL (Frontend §4.5) e **RF.19** rende la validazione dell'URL una responsabilità di sistema: il frontend non è un confine di validazione, quindi è il backend a ricevere l'URL e a derivarne **repoOwner** e **repoName**, che continua a esporre invariati nella risposta.
- **branch** e **commitSha?** al posto del campo unico **ref** (**RF.17**). Un solo campo può contenere un branch *oppure* un commit, mai entrambi insieme: non può quindi rappresentare “il branch X ancorato al commit Y”, che è esattamente ciò che **RF.17** chiede e che **RF.22** deve poi verificare. Frontend §4.5 conferma la separazione, con due campi distinti.
- **sprintId** non compare più. Nella PoC era un campo opzionale del contesto; nell'MVP lo Sprint ID è richiesto in modo interattivo durante l'esecuzione (§9.2.2) e, una volta fornito dall'utente, persistito come nuovo campo opzionale su *Task stesso*, non su *AnalysisContext*. La collocazione è più corretta perché un contesto è condiviso da più operazioni in un unico avvio, mentre lo Sprint ID riguarda le sole operazioni Changelog.

- Il significato di `paths` per `scopeType`: `DIRECTORIES` è ora definito esplicitamente (**RF.28**): la PoC dichiarava il valore ma non la sua semantica di espansione (§6.5).

Il campo `branch` compare anche nella risposta, dove la PoC non esprimeva alcun riferimento: serve all'intestazione del report, che deve mostrare il riferimento di base impiegato (**RF.56**).

6.3 Sequenza di validazione

I controlli su riferimento e ambito non sono indipendenti fra loro, ma i passi ordinati di un'unica procedura eseguita da `POST /contexts`. Nulla viene persistito finché tutti non sono superati; il primo che fallisce interrompe la creazione e restituisce l'errore corrispondente.

1. **Validazione sintattica dell'URL** (**RF.19**, *nuovo*) — dichiarata sul DTO ed eseguita prima di qualunque chiamata in uscita, così un URL malformato viene respinto con 400 senza consumare una richiesta verso GitHub. Il controllo vincola anche l'host a GitHub: essendo GitHub l'unico provider supportato (**RF.11**), un URL rivolto altrove viene respinto qui, invece di fallire più tardi con un fuorviante “repository non trovato”.
2. **Estrazione di `repoOwner` e `repoName`** (**RF.16**, *nuovo*) — dall'URL appena validato.
3. **Raggiungibilità e accesso** (**RF.20**, *nuovo come passo esplicito*) — una lettura dei metadati del repository attraverso la facade, con la credenziale dell'utente (§4). L'esito fornisce anche `isPrivate`, che è ciò che soddisfa **RV.4**: un repository privato è raggiungibile perché la chiamata viaggia sempre con il token dell'utente. In caso di fallimento l'errore è unico e volutamente combinato — repository inesistente oppure non accessibile con la credenziale configurata — perché GitHub risponde deliberatamente 404, e non 403, per un repository privato che il token non può vedere: dall'API i due casi sono indistinguibili.
4. **Esistenza del branch** (**RF.21**, *modificato*) — il backend recupera l'elenco dei riferimenti tramite `listRefs` e verifica che il branch indicato sia presente; se non lo è, il contesto viene respinto. Il metodo esiste già nella facade, ma la PoC e la Progettazione Frontend lo citano per l'autocompletamento del campo `branch`: usarlo per *verificare* un branch inviato è un impiego nuovo di un metodo esistente, non una funzionalità già progettata a questo scopo.
5. **Appartenenza del commit** (**RF.22**, *nuovo*) — solo se `commitSha` è stato fornito: si verifica che il commit esista e sia raggiungibile dallo HEAD del branch, già ottenuto al passo precedente. È il controllo che richiede la nuova voce in whitelist descritta in §5.
6. **Ancoraggio al commit** (**RF.17**) — se `commitSha` è assente, `resolvedSha` assume lo HEAD del branch, letto dal risultato del passo 4 senza una seconda chiamata; se è presente, `resolvedSha` è il commit stesso. In entrambi i casi il contesto resta

ancorato a un unico commit, il che è ciò che rende un report riproducibile anche dopo nuovi commit sul repository (PoC §4.3).

7. **Rilevamento dei linguaggi (RF.24, RV.7, *riuso*)** — si veda §6.4.
8. **Ambito non vuoto (RF.29, *nuovo*)** — per `scopeType` `FILES` o `DIRECTORIES`, `paths` deve contenere almeno un elemento dopo normalizzazione; altrimenti il contesto è respinto (§6.5.2).
9. **Esistenza dell'ambito (RF.30, *nuovo*)** — ogni percorso di `paths` viene verificato contro l'albero del repository letto a `resolvedSha` (§6.5.2).
10. **Persistenza (RF.15, *riuso*)** — l'`AnalysisContext` viene salvato e il `contextId` restituito. Resta invariata la scelta della PoC di persistere il contesto invece di passarlo inline: più Task dello stesso avvio lo condividono senza duplicarlo.

6.4 Linguaggi supportati

Invariato rispetto alla PoC, che già applica “un filtro sui linguaggi supportati (TypeScript, JavaScript, Python) [...] alla costruzione del contesto d'analisi” (§3) tramite `detectLanguages()` e `assertSupportedLanguages()` (§6, fig. 5). L'insieme supportato (**RV.7**) vive come elenco lato backend, così estenderlo in futuro è una modifica di configurazione e non una riprogettazione.

Il filtro agisce in fase di costruzione del contesto, non come marcatura a valle: i file in linguaggi non supportati non entrano mai nel materiale inviato a un agente. `detectedLanguages` compare in due risposte: quella dell'endpoint di navigazione dell'albero (§8.1), perché l'avviso **non bloccante** di **RF.24** è mostrato sopra l'albero mentre l'utente seleziona, quindi prima che `POST /contexts` venga chiamata; e `AnalysisContextDto`, dove resta persistito sul contesto.

RV.8 — lingua naturale non inglese — riguarda due superfici distinte, con proprietari diversi. Per il contenuto del repository: durante `POST /contexts`, insieme a `detectLanguages`, un controllo leggero (una libreria di rilevamento lingua, non il modello) analizza un estratto del README letto tramite la facade; un risultato non inglese produce lo stesso avviso non bloccante di **RF.24**. Per il contenuto del Changelog: nessun controllo lato backend — **RF.95** stabilisce già che il backend non vede mai i singoli campi delle Issue, quindi un controllo sulla lingua dei titoli può avvenire solo dentro il loader dell'agente, che lo segnalerebbe come qualunque altro riscontro proprio (allo stesso modo di **RF.100**).

6.5 Ambito di analisi

L'ambito di analisi è la seconda metà del contratto di creazione del contesto (`scopeType` e `paths` in `CreateContextDto`, §6.2), validato e persistito nella stessa chiamata `POST /contexts` che risolve il riferimento di base — non un passo separato né una risorsa a sé. La sua validazione occupa gli ultimi passi della sequenza di §6.3, dopo il rilevamento dei linguaggi e prima della persistenza.

6.5.1 I tre tipi di ambito

`scopeType` assume tre valori, invariati dalla PoC: `FULL_REPOSITORY`, `FILES`, `DIRECTORIES`.

- **FULL_REPOSITORY (RF.26)** — l'intero repository, `paths` omesso; invariato dalla PoC (§7.4). La selezione è sempre disponibile: nessuna configurazione la disabilita e nessun controllo dimensionale la blocca alla creazione del contesto (§6.5.3).
- **FILES (RF.27)** — uno o più percorsi di file, elencati in `paths`. È la rappresentazione univoca: un percorso significa esattamente quel file, senza alcuna regola di espansione da definire. `paths` viene normalizzato prima di ogni validazione (nessuno slash iniziale, nessun segmento `./` o `..`, duplicati eliminati), cosicché due modi di scrivere lo stesso file non possano contare due volte.
- **DIRECTORIES (RF.28)** — una o più directory, ancora in `paths`, ma con una semantica che la PoC non definiva: ciascun percorso è un prefisso, e comprende ricorsivamente ogni file al di sotto, a qualunque profondità. L'espansione avviene alla lettura, mai alla scrittura: il contesto memorizza il prefisso scelto dall'utente, non l'elenco espanso, perché l'ancoraggio a `resolvedSha` (RF.17) garantisce che espandere ora o più tardi produca lo stesso risultato.

Una selezione mista — alcune directory intere insieme a singoli file, o una directory con un figlio deselezionato — non è rappresentabile da un singolo `scopeType`: l'albero a tre stati di Frontend §4.5 può produrre esattamente questo caso. Si ricade allora su `FILES`, con l'elenco esplicito dei file sopravvissuti alla selezione: l'albero già renderizzato dal frontend possiede quelle foglie per il proprio contatore in tempo reale, quindi la conversione non costa alcuna chiamata aggiuntiva. Estendere l'enumerazione con un tipo per voce è stata una possibilità scartata, come modifica di contratto non necessaria nell'MVP.

6.5.2 Validazione dell'ambito

Due controlli, entrambi in `POST /contexts` prima della persistenza.

Ambito non vuoto (RF.29) — per `FILES` o `DIRECTORIES`, `paths` deve contenere almeno un elemento dopo normalizzazione e deduplicazione, altrimenti 400; vale anche il caso simmetrico, `paths` con `FULL_REPOSITORY`. Validazione dichiarativa pura, prima di qualunque chiamata a GitHub.

Esistenza dell'ambito (RF.30) — una volta ancorato il riferimento a `resolvedSha` (RF.17), il backend legge una sola volta l'albero del repository a quel commit e verifica ogni percorso di `paths` per appartenenza a quell'insieme. La stessa lettura fornisce il tipo di ciascuna voce: un ambito `FILES` che punta a una directory (o il contrario) è respinto come percorso inesistente; i collegamenti a submodule non sono selezionabili.

Il controllo è necessario perché la navigazione dell'albero (`GET /repositories/tree`, §8.1) e la conferma sono chiamate separate: se il branch avanza nell'intervallo tra le due, un percorso navigato può non esistere più alla conferma. Non si presenta con un `commitSha` esplicito, perché le due chiamate risolvono allo stesso commit. È un test di appartenenza su un'unica lettura, non una chiamata per percorso: la cache Redis (§5,

chiave `repo@sha:path`) rende gratuita la lettura quando navigazione e conferma risolvono allo stesso commit.

6.5.3 Limite dimensionale

Il guardiano dimensionale (**RF.31**) resta invariato e lato agente Python, nel nodo *componi prompt* del grafo LangGraph (PoC §4.3): il backend non introduce un controllo proprio. Se l'ambito eccede la capacità del modello, la Task fallisce ed è gestita dal percorso di errore a runtime già usato per ogni altro fallimento di esecuzione, con reindirizzamento a `/select` per ridurre l'ambito.

`estimatedFileCount`, già su `AnalysisContextDto`, resta un valore puramente indicativo per il contatore del frontend (Frontend §4.5, UC19), non un input di alcun controllo backend.

7 Catalogo delle Operazioni e Avvio

Due momenti della stessa schermata (Frontend §4.6): il backend espone l'elenco delle operazioni che l'utente può avviare, e riceve poi la selezione confermata.

La PoC progetta già entrambi gli endpoint; l'MVP definisce il tipo che il primo restituisce e aggiunge i controlli che il secondo esegue prima di accodare le operazioni.

7.1 Il catalogo delle operazioni

`GET /operations` è invariato dalla PoC (§7.2). Ciò che manca è il tipo che restituisce: `OperationDescriptorDto` è nominato nel contratto (§4.2) ma non compare fra gli schemi di §7.4. L'MVP lo definisce:

```
interface OperationDescriptorDto {
    code: OperationCode           // identificativo stabile, invariato
    displayName: string           // NUOVO: etichetta mostrata a schermo
    description: string           // NUOVO: descrizione sintetica
    agent: 'DOCS' | 'SECURITY' | 'CHANGELOG'
}
```

`code` e `displayName` restano distinti perché servono a due scopi diversi: il primo è l'identificativo che attraversa API e persistenza (`Task.operation`, `Report.operation`) e non cambia mai; il secondo è testo di interfaccia, riscrivibile senza invalidare le Task e i Report già archiviati.

L'elenco non è una lista mantenuta a parte, ma è costruito dalle voci di **AgentRegistry** (**RF.40**): un'operazione può comparire nel catalogo solo se il backend sa instradarla. `displayName` e `description` sono testo fisso associato a ciascuna voce del registro, servito in italiano, la lingua dell'interfaccia.

La risposta contiene solo le operazioni permesse al ruolo del chiamante, letto dal claim del JWT (§3, **RF.7**). La mappatura ruolo→operazione non viene ridefinita qui: resta quella di Frontend Tabella 2, dove un'operazione può essere permessa a più ruoli.

7.2 Avvio di un batch

POST /tasks e CreateTaskBatchDto sono riutilizzati invariati dalla PoC (§7.2, §7.4), inclusa la risposta 202 con {taskIds, batchId} e il ritorno immediato in stato PENDING imposto da ADR-3. Configurazione e avvio sono la stessa chiamata: non esiste una risorsa intermedia, e nulla viene persistito se l'avvio non riesce.

Selezionare più operazioni insieme (**RF.37**) e avviarle insieme (**RF.39**) non richiedono nulla di nuovo: `operations: OperationCode[]` accetta già più codici, e ogni voce diventa una Task indipendente sotto un unico `batchId`, che è anche la chiave su cui `batch.completed` riporta l'esito complessivo (**RF.47**). Quante di quelle Task vengano poi eseguite davvero in parallelo non dipende dal backend ma dal servizio agenti, che è un servizio ECS distinto: Progettazione AWS §4.1 dimensiona entrambi a una sola istanza (`Desired Count: 1`), quindi nell'MVP un solo agente per volta è effettivamente in esecuzione.

L'array viene normalizzato prima dell'uso (**RF.36**): i valori che non appartengono all'enumerazione chiusa sono respinti con 400, e i duplicati eliminati, così la stessa operazione non può produrre due Task identiche nello stesso batch.

7.3 Controlli prima dell'accodamento

Tre controlli, tutti eseguiti prima che una qualunque Task entri in coda. Nessuno accoda parzialmente: o l'intero batch parte, o non parte nulla.

Selezione non vuota (RF.41) — dopo la normalizzazione, `operations` deve contenere almeno un elemento, altrimenti 400. È il controllo di **RF.29** un passo più avanti: là un ambito vuoto alla creazione del contesto, qui una selezione vuota all'avvio.

Prerequisiti (RF.42) — il `contextId` deve esistere e appartenere al chiamante, e una credenziale GitHub deve essere configurata. Sono verifiche di sola esistenza: lo scope del token è già stato verificato al salvataggio della credenziale (**RS.4**) e ripeterlo a ogni avvio costerebbe una chiamata esterna prima di ogni Task, contro il budget di **RQ.10**. Il controllo di appartenenza del contesto non è una formalità: senza, un `contextId` indovinato permetterebbe di avviare operazioni sul repository di un altro utente.

Il requisito sembra chiedere prerequisiti diversi per ciascuna operazione, ma nell'MVP l'insieme è uniforme, per decisioni prese altrove: lo Sprint ID, l'unico parametro specifico di un'operazione, è raccolto durante l'esecuzione e non prima (§9.2.2, **RF.98–RF.99**), e `POLICY.md`, che serve a `SECURITY_POLICY`, è un file del repository: la sua assenza emerge come fallimento a runtime (**RF.70**), non come blocco preventivo.

Ruolo (RF.7) — se anche una sola operazione selezionata non è permessa al ruolo del chiamante, l'intero batch è respinto con 403.

8 Contratto API

8.1 API REST pubbliche

Tutti i path sono relativi alla base `/api/v1` ed esposti in Swagger, come nella PoC (§7.2). Le chiamate autenticate portano il JWT nell'header `Authorization: Bearer.`

8.1.1 Endpoint riusati invariati

Riusati dalla PoC (§7.2) senza modifiche a firma, DTO o semantica; non compaiono nella tabella seguente:

- GET `/auth/me` → `UserProfileDto` — profilo dell'utente autenticato.
- GET `/auth/health` — health check consumato dall'infrastruttura (**RV.15**).
- GET `/credentials` → `ServiceCredentialDto[]` — credenziali configurate (**RF.12**).
- DELETE `/credentials/:id` — revoca locale di una credenziale (**RF.12**).
- GET `/repositories` → `RepositorySummaryDto[]` — repository accessibili con la credenziale dell'utente (**RF.15**).
- POST `/tasks/:id/cancel` — annullamento di una Task (**RF.46**).

8.1.2 Endpoint nuovi o modificati

Tabella 2: Endpoint REST pubblici nuovi o modificati rispetto alla PoC.

Metodo e path	Scopo	Requisiti
POST <code>/auth/register</code>	Registrazione. Body <code>RegisterDto</code> , risposta 201 <code>UserProfileDto</code> senza token. Nella PoC era uno stub: validazione, hashing e persistenza sono nuovi.	RF.1–RF.6, RS.1
POST <code>/auth/login</code>	Autenticazione. Body <code>LoginDto</code> , risposta <code>AuthTokenDto</code> (JWT HS256, scadenza 8 ore). Stub nella PoC.	RF.7–RF.9
POST <code>/auth/logout</code>	Nuovo. Nessun effetto lato server: il JWT è stateless, risponde 204. Esiste perché il comando “Esci” del frontend prevede uno stato esplicito di fallimento di rete.	—

(continua nella pagina successiva)

(segue dalla pagina precedente)

Metodo e path	Scopo	Requisiti
POST /credentials	Salva la credenziale cifrata. Body <code>CreateCredentialDto</code> , risposta <code>ServiceCredentialDto</code> . Modificato: controllo di formato e chiamata di test a GitHub precedono la persistenza, che non avviene se falliscono.	RF.10– RF.14, RS.2
POST /credentials/:id/ validate	Ri-valida una credenziale già salvata. Firma invariata; nell'MVP è il secondo dei due momenti di validazione, non l'unico.	RF.12, RS.4
GET /repositories/refs	Branch disponibili per l'autocompletamento. Query: <code>repoUrl</code> . Modificato: sostituisce <code>/repositories/ :owner/:repo/refs</code> , che presume l'URL già separato.	RF.16– RF.17
GET /repositories/tree	Struttura del repository per l'albero di selezione. Query: <code>repoUrl</code> , <code>branch</code> , <code>commitSha?</code> . Risposta <code>RepositoryTreeDto</code> : <code>entries</code> (percorso e tipo) e <code>detectedLanguages</code> , calcolato sull'albero appena letto per l'avviso non bloccante mostrato durante la selezione (§6.4). Modificato con la stessa sostituzione di firma.	RF.24, RF.25– RF.28
POST /contexts	Crea l' <code>AnalysisContext</code> . Body <code>CreateContextDto</code> , risposta <code>AnalysisContextDto</code> (§6.2). Modificato nel corpo; esegue la sequenza di validazione di §6.3.	RF.15– RF.31
GET /operations	Catalogo delle operazioni permesse al ruolo del chiamante. Risposta <code>OperationDescriptorDto[]</code> , tipo che la PoC nominava senza definirlo.	RF.32– RF.34
POST /tasks	Avvia un batch. Body <code>CreateTaskBatchDto</code> , risposta 202 { <code>taskIds</code> , <code>batchId</code> }. Contratto invariato; nuovi i controlli su selezione vuota e prerequisiti prima dell'accodamento.	RF.35– RF.42
GET /tasks?{query} GET /tasks/:id	Elenco delle Task per la dashboard e dettaglio della singola Task. Entrambi restituiscono lo stesso <code>TaskDto</code> (PoC §7.4), esteso col campo nullable <code>pendingInput</code> (§9.2.2): la dashboard lo riceve già nell'elenco, senza interrogare le Task una per una.	RF.43, RF.45

(continua nella pagina successiva)

(segue dalla pagina precedente)

Metodo e path	Scopo	Requisiti
POST /tasks/:id/input	Nuovo. Consegna la risposta dell'utente a una sospensione in corso e ne provoca la ripresa (§9.2.2).	RF.98– RF.105
GET /reports	Storico dei report del chiamante, filtrabile per <code>operation</code> , <code>from</code> e <code>to</code> (PoC §7.2). Risposta <code>ReportSummaryDto[]</code> : proiezione di sintesi con <code>title</code> (§12.7), esclusi <code>body</code> , <code>proposal</code> ed <code>error</code> . Nuovo l'ambito per utente.	RF.49– RF.52
GET /reports/:id	Dettaglio di un report. Risposta <code>ReportDto</code> (§12.2), estesa con <code>title</code> , <code>context</code> , <code>executionTimeMs</code> , <code>pullRequestUrl</code> e <code>pendingAction</code> . Nuovo il controllo di proprietà: 404, non 403, se il report non appartiene al chiamante.	RF.53– RF.64
GET /reports/:id/ export?format=pdf	Esporta il report in PDF, composto programmaticamente dal <code>ReportDto</code> persistito e restituito in streaming (§12.8). Stesso percorso della PoC (<code>?format=md</code>); il formato Markdown decade, <code>pdf</code> è l'unico valore ammesso. Stessa regola di proprietà di GET /reports/:id.	RF.73– RF.74

GET /repositories/refs, GET /repositories/tree e POST /contexts condividono un'unica routine di parsing dell'URL e verifica di raggiungibilità (**RF.19–RF.20**), invocata da ciascuna per proprio conto: nessuna riusa il risultato di una chiamata precedente.

L'identificativo dell'operazione in elaborazione (**RF.45**) è già esposto dal campo `operation: OperationCode` presente su entrambi gli schemi `Task`; nome e descrizione leggibili provengono da GET /operations.

8.2 Canale realtime (WebSocket)

8.2.1 Eventi di avanzamento e completamento (RF.44, RF.47)

Il canale WebSocket (§7.3 della PoC) resta il meccanismo riusato per intero, con una precisazione sul modello di sottoscrizione. L'MVP adotta un modello di sottoscrizione per utente anziché per Task: alla connessione, il socket viene automaticamente iscritto a una stanza `user:<userId>` in base al JWT trasmesso in fase di handshake, e riceve così tutti gli eventi relativi a ogni Task di sua proprietà. Questo rende superfluo un evento di sottoscrizione esplicito lato client per singolo `taskId`.

I primi quattro eventi sono riusati identici dalla PoC (§7.3). Per **RF.47**: la PoC associa esplicitamente `batch.completed` a questo requisito, ma il segnale più preciso — a livello di singola Task, non di batch — è già il campo `reportId` di `TaskDto` (§2); poiché `reportId`

Evento	Payload
<code>task.progress</code>	<code>{taskId, stage, percent}</code>
<code>task.updated</code>	<code>{taskId, status, reportId?}</code>
<code>task.failed</code>	<code>{taskId, error}</code>
<code>batch.completed</code>	<code>{batchId, completed, failed}</code>
<code>task.inputRequired</code>	<code>{taskId, kind, taskIds?, technicalReportId?}</code>

Tabella 3: Eventi WebSocket emessi dal backend verso il frontend; i primi quattro riusati identici dalla PoC (§7.3), l'ultimo nuovo per l'MVP (§9.2.2).

è valorizzato anche sulle Task **FAILED** (§11), il segnale di completamento con esito positivo è la condizione `status === 'COMPLETED'` congiunta a `reportId` non nullo, non la sola presenza di `reportId`. Entrambi i segnali restano disponibili, coprendo i due livelli di granularità con cui il requisito può presentarsi. `task.inputRequired` è l'unico evento nuovo della release (§9.2.2).

Payload di `task.inputRequired` — forma piatta, coerente con quella degli altri quattro eventi. `taskId` è un'aggiunta necessaria rispetto al solo valore di `pendingInput`: senza, il frontend non saprebbe a quale Task riferire l'evento con più di una in corso. Il campo è denominato `technicalReportId`, non `reportId`, per evitare una collisione con l'omonimo campo già usato da `task.updated` e da `TaskDto.reportId` per il report finale della Task.

8.3 API interne (agente → backend)

[Da completare — riuso da PoC §7.5]

L'endpoint `POST /internal/tasks/:id/progress`, usato dall'agente per riportare l'avanzamento a metà esecuzione, è implementato. Autenticato con lo stesso `InternalAuthGuard` a HMAC-SHA256 usato dalla facade GitHub (§5), aggiorna `progressPercent/currentStage` sulla Task e inoltra l'evento `task.progress` alla stanza dell'utente proprietario.

9 Orchestrazione ed esecuzione

9.1 Riuso dalla PoC

L'instradamento delle operazioni verso gli agenti e il ciclo di vita delle Task non vengono riprogettati per l'MVP: la PoC (§8) descrive già un orchestratore **interamente valido per l'MVP**, non un suo sostituto provvisorio.

Restano quindi invariati:

- L'instradamento deterministico operazione → agente tramite `AgentRegistry`, una mappa statica popolata al bootstrap, senza alcuna inferenza da parte di un modello linguistico (**RV.1**).

- L'esecuzione asincrona su coda: `POST /tasks` accoda il lavoro e risponde immediatamente con lo stato `PENDING`; un worker (`TaskProcessor`) esegue la Task separatamente, isolando i fallimenti di un job dagli altri (**RF.48**).
- Il ciclo di vita a cinque stati (`PENDING`, `RUNNING`, `COMPLETED`, `FAILED`, `CANCELLED`), con le transizioni ammesse imposte da `Task.canTransitionTo()` (**RF.46**).
- L'avanzamento in tempo reale via WebSocket, coerente con l'esecuzione asincrona (**RF.44**).

9.2 Estensioni per l'MVP

9.2.1 Catalogo completo delle operazioni (**RF.40**)

La PoC implementava solo tre `OperationCode` (uno per agente); l'MVP promuove a obbligatorie tutte le operazioni elencate in §1.2, per un totale di sette valori (`DOCS_README`, `DOCS_INLINE`, `DOCS_API`, `SECURITY_OWASP`, `SECURITY_POLICY`, `CHANGELOG_TECHNICAL`, `CHANGELOG_BUSINESS`). Il tipo `OperationCode` era già definito con questi sette valori nel contratto della PoC: l'estensione consiste nel registrare in `AgentRegistry` le quattro voci mancanti, non nel modificare il meccanismo di instradamento.

9.2.2 Sospensione per input dell'utente (**RF.43**, **RF.98–RF.105**)

Le operazioni dell'Agente Changelog richiedono, a differenza di Docs e Security, informazioni che l'utente non può fornire prima dell'avvio. Per questo la Task guadagna un nuovo campo sul proprio stato: `pendingInput`. È una condizione sospensiva interna allo stato `RUNNING`, non un sesto stato — coerentemente col modello già adottato dal documento di Progettazione Frontend, che la introduce nella stessa forma.

Il campo è *nullable*, non opzionale: è sempre presente in ogni `TaskDto`, e vale `null` quando la Task non è in attesa di nulla. Questa scelta riprende lo stesso pattern già usato da tre campi del contratto esistente (`currentStage`, `reportId`, `error` — tutti `T | null`), così da mantenere un'unica convenzione nell'intero DTO invece di introdurne una seconda.

È un tipo a discriminante (`kind`), lo stesso pattern già usato da `Block` nel contratto `Report` della PoC (§5):

```
type PendingInput =
  | { kind: 'SPRINT_ID' }
  | { kind: 'INCOMPLETE_TASKS'; taskIds: string[] }
  | { kind: 'BUSINESS_CONFIRMATION'; technicalReportId: string }
  | null;
```

Le tre varianti corrispondono a tre distinti punti di sospensione, tutti risolti dallo stesso endpoint `POST /tasks/:id/input`:

1. `SPRINT_ID` — richiesta dello Sprint ID (**RF.98–RF.99**), per entrambe le operazioni Changelog. Il valore ricevuto da `POST /tasks/:id/input` deve essere non vuoto,

altrimenti 400; se corrisponda a una milestone reale emerge solo quando l'agente la interroga (**RF.70**).

2. **INCOMPLETE_TASKS** — segnalazione degli elementi con metadati insufficienti (**RF.100–RF.102**); attivata solo se ne viene rilevato almeno uno da escludere, altrimenti saltata. Nome e payload (`taskIds: string[]`) sono quelli della Progettazione Frontend (§7.4): l'elemento validato non è un **Task** del backend, è la User Story o il task dello Sprint recuperato da GitHub Issues (§1.2.3) — l'omonimia con l'entità del backend è un rischio di lettura noto e accettato per restare allineati al contratto del Frontend.
3. **BUSINESS_CONFIRMATION** — conferma del changelog tecnico prima di procedere alla generazione di business (**RF.103–RF.105**); attivata solo per **CHANGELOG_BUSINESS**, mai per **CHANGELOG_TECHNICAL**. È il changelog *tecnico* a essere confermato o rifiutato in questo passo, non quello di business, che non esiste ancora a questo punto dell'esecuzione. Porta `technicalReportId` (Frontend §7.4), non il testo in linea: la pagina di dettaglio ha bisogno di un identificativo da aprire, e il testo dell'anteprima sarebbe comunque una duplicazione del report tecnico già persistito (§12.6).

Chi determina la sospensione. Per Docs e Security il backend invoca l'agente Python con una singola chiamata, come nella PoC. Per Changelog, il filtro di qualità sui metadati è già interamente risolto dalla PoC nel **ChangelogLoader** dell'agente (§9.3, “procedi escludendo”), che oggi esclude e prosegue senza intervento umano. L'MVP riusa quella logica invariata e aggiunge un solo elemento: la capacità di sospendersi in quel punto invece di procedere da sola. La responsabilità si divide così:

- **SPRINT_ID** — determinata dal **backend**, prima di invocare l'agente: se lo Sprint ID non è noto, l'agente non viene avviato (coerente con **RF.99**; nella PoC era fornito a priori, §7.4).
- **INCOMPLETE_TASKS** — determinata dall'**agente**: il **ChangelogLoader**, invariato, si sospende invece di escludere e proseguire da solo, restituendo al backend l'elenco degli elementi insufficienti; il backend si limita a valorizzare `pendingInput` e a inoltrare la decisione dell'utente, senza reimplementare la validazione.
- **BUSINESS_CONFIRMATION** — determinata dal **backend**: nessuna logica PoC da preservare, poiché il changelog di business non era previsto (§9.3, “Non implemento”).

Il meccanismo interno di sospensione/ripresa dell'agente (es. l'interruzione nativa di LangGraph) è una scelta del servizio agenti, fuori dal perimetro di questo documento; qui rileva solo il contratto tra i due servizi. In ogni caso è sempre e solo il backend a scrivere `pendingInput` e a emettere `task.inputRequired`: l'agente non comunica mai direttamente col frontend.

Questa interattività è il rischio architetturale alto, non validato dalla PoC, individuato dalla Progettazione Frontend (§12, Tabella 10): in caso di tempo insufficiente, il fallback dichiarato è il comportamento autonomo della PoC, sacrificando temporaneamente **RF.98–RF.102**.

Evento/Endpoint	Direzione	Ruolo
<code>task.inputRequired</code>	backend → frontend	Notifica che <code>pendingInput</code> è stato valorizzato; il payload coincide col nuovo valore del campo.
POST <code>/tasks/:id/input</code>	frontend → backend	Consegna la risposta dell'utente (lo Sprint ID, la scelta di proseguire/annullare, la conferma/rifiuto) affinché il backend la usi per la fase successiva — invocando l'agente per la prima volta, riprendendone l'esecuzione sospesa, o terminando la Task, a seconda di quale sospensione viene risolta. <code>pendingInput</code> torna a <code>null</code> come conseguenza della ripresa, non come effetto diretto della chiamata.

Tabella 4: Meccanismo condiviso di sospensione/ripresa della Task Changelog.

Se l'utente rifiuta la conferma `BUSINESS_CONFIRMATION` (**RF.105**), lo stato finale della Task è `CANCELLED`: una Task `CHANGELOG_BUSINESS` è, per definizione, quella che promette un changelog di business, indipendentemente dal fatto che il testo tecnico intermedio sia già stato generato e mostrato in anteprima. Chi desidera solo il changelog tecnico avvia direttamente un'operazione `CHANGELOG_TECHNICAL`.

10 Dettaglio degli Agenti

Questa sezione descrive, operazione per operazione, il “come” promesso in §1.2: non che cosa fa l'agente al proprio interno — quello resta al servizio agenti, fuori dal perimetro di questo documento — ma che cosa fa il backend prima e dopo averlo invocato.

10.1 Agente Docs

L'Agente Docs è l'unico dei tre che produce un file potenzialmente già presente nel repository, ma il vincolo di sola lettura non ne risente: “l'agente legge dal repository ma non ci scrive” (PoC §6.4). In scrittura produce solo una `Proposal` — un diff verso il file di destinazione, persistito come Report — e resta il backend a scrivere davvero su GitHub, dopo l'esecuzione dell'agente e mai durante (**ADR-1**, **ADR-4**).

10.1.1 Generazione del README (**RF.82**)

POST `/tasks` instrada `DOCS_README` tramite `AgentRegistry` all'Agente Docs (**RF.40**, §9.2.1). Il backend invoca l'agente con il contesto selezionato e attende la sua `Proposal`; ricevuta, la persiste come corpo del Report, apre la Pull Request e ne scrive l'URL (**ADR-4**,

RF.63, §??). Un fallimento dell'apertura è già catalogato come `PR_CREATION_FAILED` (**RF.72**, §11.5).

Non c'è alcun meccanismo nuovo in questo passo: è la stessa sequenza `invoca`→`ricevi`→`apri PR`→`salva URL` già progettata in generale per ogni operazione con `Proposal`. Ciò che è nuovo è solo l'operazione stessa e la voce corrispondente in `AgentRegistry`. Un README non ancora esistente non è un caso speciale: `diffUnified` è un diff unificato ordinario, che rappresenta un file nuovo come diff contro il vuoto senza bisogno di alcuna estensione dello schema.

10.1.2 Documentazione inline: generazione e correzione (**RF.83**, **RF.84**)

Stesso meccanismo di RF.82, con un'unica differenza: l'operazione è `DOCS_INLINE` (già nel PoC), e ciò che il backend riceve come `Proposal` è un diff di commenti JSDoc o docstring anziché di README.

La PoC copre già il caso dei commenti assenti, per intero (§9.1, “Implemento”). **RF.84** estende la stessa operazione ai commenti *disallineati* dal codice, non solo a quelli assenti. Per il backend questo non introduce un secondo passo: **RF.83** e **RF.84** condividono lo stesso `OperationCode`, la stessa voce di `AgentRegistry` e lo stesso ciclo `invoca`→`ricevi`→`apri PR`. Se il diff che l'agente produce corregga un commento sbagliato o ne scriva uno assente è una decisione interamente interna all'agente durante quella stessa invocazione.

Segnalazione di complessità eccessiva (RF.85, desiderabile). Anche questo già “Implemento” nella PoC: “dove il codice è troppo intricato, l'agente emette un avviso strutturato invece di forzare un commento sbagliato” (§9.1). Lo schema di quell'avviso, `ComplexityWarningBlock`, è già definito in **RF.61** (§12) e si applica alle sole `DOCS_INLINE` e `DOCS_API`, mai a `DOCS_README`, che documenta il progetto e non una singola unità di codice. Il backend non aggiunge alcun controllo proprio: riceve il blocco come parte del Report e lo persiste così com'è.

10.1.3 Documento API (**RF.86**)

Stesso meccanismo di RF.82: nuova operazione (`DOCS_API`), nuova voce in `AgentRegistry`, stessa `Proposal`→`PR`→`URL`. La PoC non la implementa (§9.1, stesso elenco di RF.82), e che cosa costituisca “informazione sugli Endpoint” e come venga estratta dal codice sorgente è una decisione del servizio agenti, non vincolata da questo documento.

10.2 Agente Security

A differenza di Docs, l'Agente Security non scrive mai: resta *flag-only*, individua e documenta, ed è il Security Auditor a intervenire manualmente sul codice, al di fuori di Code Guardian. Non esiste quindi alcuna apertura di Pull Request per queste operazioni: il backend invoca l'agente e persiste il suo Report così com'è.

10.2.1 Scansione OWASP (RF.87)

POST /tasks instrada SECURITY_OWASP tramite AgentRegistry all'Agente Security (RF.40). A differenza delle operazioni Docs, il backend non apre alcuna Pull Request: invoca l'agente e persiste il Report ricevuto — un elenco di FindingBlock — così com'è. La scansione è già interamente coperta dalla PoC (§9.2).

10.2.2 Classificazione e localizzazione dei riscontri (RF.88–RF.91)

Categoria, gravità, file, righe e rimedio di ogni riscontro non sono un nuovo schema: sono gli stessi campi che FindingBlock già porta per RF.62 (§??) — category, severity, filePath, lineStart/lineEnd? e remediation come SNIPPET o TEXT. Il backend riceve e persiste questi valori così come arrivano, senza derivarli né correggerli.

RF.90 e RF.91 sono i due rami dello stesso campo remediation, già definito da RF.62: quale dei due venga usato per un dato riscontro è una decisione dell'agente, non del backend.

10.2.3 Conformità a POLICY.md (RF.92)

Stessa meccanica di RF.87, instradata su SECURITY_POLICY anziché SECURITY_OWASP: nuova voce in AgentRegistry, nessuna differenza nel modo in cui il backend invoca l'agente o persiste il Report.

L'assenza di POLICY.md non è un caso da progettare qui: è l'esempio che RF.70 già nomina per una risorsa di contesto mancante (§11), rilevata dall'agente e propagata dal backend come ogni altro fallimento della stessa famiglia.

10.3 Agente Changelog

Come Security, anche Changelog non apre mai Pull Request: il suo output è testo, non un cambiamento al repository.

10.3.1 Changelog tecnico (RF.94–RF.96)

POST /tasks/:id/input risolve la sospensione SPRINT_ID (§9.2.2), e lo Sprint ID confermato è ciò che viene passato all'agente quando questo è invocato per CHANGELOG_TECHNICAL tramite AgentRegistry (RF.40). Il backend persiste ed espone il Report ricevuto — un unico TextBlock in Markdown — senza alcun passo di scrittura successivo. La lettura delle Issue, l'estrazione dei metadati e la composizione del testo sono già interamente coperte dalla PoC (§9.3).

10.3.2 Metadati insufficienti (RF.100–RF.102)

Già interamente progettato in §9.2.2 come sospensione WORK_ITEM_QUALITY: il backend valorizza pendingInput con l'elenco ricevuto dall'agente, e POST /tasks/:id/input risolve la sospensione riprendendo l'agente per procedere escludendo, oppure annullando la Task (CANCELLED).

10.3.3 Changelog di business (RF.103–RF.105)

Stessa meccanica di sospensione, variante `BUSINESS_CONFIRMATION` (§9.2.2): `CHANGELOG_BUSINESS` è un'unica Task la cui prima fase produce il changelog tecnico, poi sospesa in attesa di conferma. La conferma riprende l'agente per la fase di business; il rifiuto porta la Task a `CANCELLED` senza mai invocare quella fase.

10.3.4 Leggibilità del Changelog di business (RQ.7)

Nuovo codice d'errore, `READABILITY_TOO_LOW` (§11.2): l'agente calcola l'indice Flesch Reading Ease sul proprio output e decide da sé, dal proprio budget di tentativi, quando arrendersi. Il backend non calcola l'indice né decide quando fermarsi: riceve il codice come ogni altro errore rilevato dall'agente e propaga il fallimento allo stesso modo.

11 Gestione degli Errori e Validazione dell'Esecuzione

Questa sezione progetta **RF.65–RF.72**, **RQ.5** e **RQ.6**. Il meccanismo è interamente quello della PoC: il lavoro dell'MVP consiste quasi solo nel dare un nome ai casi che la PoC trattava in blocco.

11.1 Riuso dalla PoC

Restano invariati e non vengono ridiscussi qui: il campo `error` su `TaskDto` e sull'envelope `Report` (PoC §7.4 e fig. 4); lo stato terminale `FAILED` e l'evento `task.failed` (PoC §8.2, §7.3); l'isolamento dei fallimenti nel `TaskProcessor`, per cui un job che fallisce non trascina gli altri (PoC §8.1, **RF.48**); la semantica del rilancio, che è “una Task nuova che tiene il riferimento alla precedente” (PoC §8.2), per cui il comando “Riprova” del frontend crea una Task con `previousTaskId` e non riapre quella fallita.

RF.65 è il requisito generico (UC27) e **RF.66–RF.72** ne sono le specializzazioni: per questo ciascuna costa un valore di enumerazione, non un meccanismo.

Due precisazioni al contratto della PoC. La prima: un fallimento produce sempre *anche* un `Report` (`status: FAILED`, corpo vuoto, `error` valorizzato). È ciò che tiene attivo il comando “Visualizza report” sugli elementi falliti della dashboard (Frontend §4.7) e permette alla vista di dettaglio di sostituire i blocchi con il componente d'errore (Frontend §8.3).

La seconda ne discende: `reportId` è valorizzato quando `status` è `COMPLETED` oppure `FAILED`, e non solo a `COMPLETED` come annota la PoC (§7.4); resta `null` negli stati non terminali e in `CANCELLED`. La lettura che ne discende per **RF.47** è in §8.2.1.

11.2 Delta: il dominio di ErrorKind

La PoC chiude `ReportError.kind` su `ErrorKind = TIMEOUT | PARSING | UPSTREAM` (fig. 4), mentre `TaskError.code` resta una stringa libera (§7.4): i due lati dello stesso

errore non hanno oggi né lo stesso nome di campo né lo stesso dominio. L'MVP **non** rinomina nulla e interviene solo sul dominio, che va chiuso perché il frontend mappa ogni causa su un messaggio e un'azione secondaria diversi (Frontend §8.3, Tabella 7) e non può riconoscerla dal testo del messaggio. Il dominio è però *unico*: gli stessi valori viaggiano su `TaskError.code` e su `ReportError.kind`, che restano due nomi di campo diversi per la stessa cosa. È dal primo che il frontend legge, confrontando `error.code === 'CREDENTIAL_INVALID'` sull'evento `task.failed` (Frontend §6.4, Listing 2); uniformare i due nomi sarebbe un miglioramento, ma è una modifica al contratto della PoC che nessun requisito impone.

Tre requisiti sono già coperti dai valori esistenti; i valori nuovi sono otto, e il dominio passa da tre a undici.

Req.	ErrorKind	Stato	Rilevato da	Azione secondaria
RF.67	TIMEOUT	PoC	agente	“Riprova”
RF.68	PARSING	PoC	agente	“Riprova”
RF.65	UPSTREAM	PoC	backend	“Riprova”
RF.69	CONTEXT_TOO_LARGE	nuovo	agente	Rimanda a <code>/select</code>
RF.70	CONTEXT_RESOURCE_MISSING	nuovo	agente	Nessuna
RF.71	CONTEXT_RESOURCE_INVALID	nuovo	agente	Nessuna
RQ.8	LLM_RATE_LIMITED	nuovo	agente	<i>Da definire</i>
RQ.7	READABILITY_TOO_LOW	nuovo	agente	<i>Da definire</i>
RF.72	PR_CREATION_FAILED	nuovo	backend	Rimanda a <code>/credentials</code>
RF.66	USAGE_LIMIT_EXCEEDED	nuovo	backend	Nessuna
RF.13, RS.4	CREDENTIAL_INVALID	nuovo	backend	Rimanda a <code>/credentials</code>

Tabella 5: Dominio di `ErrorKind` nell'MVP. I messaggi mostrati all'utente non sono ridefiniti qui: restano quelli della Progettazione Frontend, Tabella 7.

Tre regole accompagnano la tabella. `UPSTREAM` è il valore di ripiego per qualunque fallimento che non corrisponda a un caso previsto, reso con il messaggio generico di Frontend §8.3.1. Un'irraggiungibilità di rete verso GitHub è sempre `UPSTREAM` e **mai** `CREDENTIAL_INVALID`, perché Frontend §13.6 chiede che la caduta del NAT Gateway non produca un falso avviso di credenziale non valida. Infine `error.stage` è valorizzato per i soli valori rilevati durante l'esecuzione di una Task: `USAGE_LIMIT_EXCEEDED` e l'occorrenza di `CREDENTIAL_INVALID` su `POST /credentials` nascono fuori da qualunque Task e viaggiano nell'involucro d'errore della risposta REST.

Degli otto valori nuovi, cinque sono rilevati dall'agente (§11.3): estendono il contratto verso il servizio agenti e vanno concordati con chi lo progetta (§11.7).

11.3 Dove vive ogni controllo

La lettura ingenua di **RF.68–RF.71** suggerirebbe di implementare le validazioni nel backend. La PoC le colloca dentro l'agente per una ragione strutturale: il *parser* è una delle tre sole cose che “implementare un agente” significa (PoC §9) ed è per forza specifico dell'agente, e il controllo dimensionale sta “lato agente Python, nel nodo componi prompt” (PoC §4.3) perché il backend non conosce la finestra di contesto del modello, che dipende dal fornitore scelto a runtime (ADR-2). Reimplementarli nel backend significherebbe fargli conoscere lo schema di output di ogni agente: esattamente ciò che l'envelope condiviso di fig. 4 esiste per evitare.

- **L'agente rileva e restituisce l'esito: RF.67–RF.71.**
- **Il backend riceve, classifica e propaga:** porta la Task a FAILED, valorizza `error`, persiste il Report fallito ed emette `task.failed`. Non ripete alcuna validazione.
- **Il backend rileva in proprio** solo ciò che nasce da lui: **RF.66** (§11.4) e **RF.72** (§11.5).

Sui ritentativi, che sono il punto in cui due meccanismi rischiano di moltiplicarsi: l'agente può ri-interrogare il modello un numero limitato di volte, ma è una scelta interna al servizio agenti e non viene replicata a livello di Task. Quando l'agente si arrende la Task è FAILED, e solo l'utente decide se rilanciare.

Due casi che sembrano errori e non lo sono. Lo Sprint ID mancante non è un fallimento: §9.2.2 lo trasforma in una richiesta interattiva, quindi **RF.70** si applica al solo Sprint ID fornito ma non corrispondente ad alcuna milestone. Allo stesso modo il cancello di qualità sui metadati delle issue (PoC §9.3) non è **RF.71** ma una sospensione (WORK_ITEM_QUALITY, **RF.100–RF.102**).

11.4 Limite di utilizzo (RF.66)

Nessuna copertura nella PoC, che non progetta alcuna contabilizzazione. Il Frontend riserva però già il comportamento d'interfaccia (Tabella 7), quindi il valore d'errore deve esistere nel contratto anche nell'ipotesi in cui il contatore fosse la prima cosa a cadere per mancanza di tempo.

Si contano le *esecuzioni*, non i token: **RF.66** parla di “chiamate”. Un tetto a token sarebbe possibile, dato che `Report` persiste già `tokensConsumed` (fig. 3), ma misurerebbe una grandezza che il requisito non nomina e che dipende dal fornitore (ADR-2). Il tetto vive in configurazione e non a database, così alzarlo non è una migrazione.

Il controllo avviene dentro `POST /tasks` **prima** dell'accodamento — “preventivamente” significa che nessun agente viene mai invocato — e respinge con 429 e `USAGE_LIMIT_EXCEEDED`. Due conseguenze vanno fissate:

- `POST /tasks` avvia più operazioni con un solo invio e restituisce `{taskIds[], batchId}` (PoC §7.2): un lotto che supera il tetto a metà è respinto **per inte-**

ro, senza accodare nulla. È atomico, e l'unico comportamento compatibile con il messaggio unico e privo di azione secondaria che il Frontend riserva a questo caso.

- All'accodamento l'unica grandezza osservabile è la Task: una Task `CHANGELOG_BUSINESS` invoca l'agente due volte (§9.2.2) e la seconda dipende da una conferma non ancora avvenuta. L'unità contata è quindi la *Task avviata*, con contatore su MongoDB accanto all'utente e finestra a mese solare, e registra ogni Task creata a prescindere dall'esito.

Concretamente, il contatore vive in una collezione dedicata `UsageCounter`, un solo documento per coppia utente/mese: `{ userId: string, yearMonth: string, count: number }`, dove `yearMonth` è la stringa "AAAA-MM" del mese solare corrente. Un indice `unique` su `(userId, yearMonth)` garantisce un solo documento per combinazione.

11.5 Fallimento dell'apertura della Pull Request (RF.72)

È l'unico requisito della sezione che riguarda un passo interamente nuovo: nella PoC non esiste (§9.1, “non implemento [...] l'apertura effettiva della Pull Request”). Nell'MVP l'apertura è un passo del backend, eseguito dopo la persistenza dell'output dell'agente (ADR-4) e dentro la stessa Task, quindi un suo fallimento fa fallire la Task: `PR_CREATION_FAILED` per un rifiuto di autorizzazione o permessi, `UPSTREAM` per un fallimento di rete (Frontend §13.6), con `error.stage` pari a `'PULL_REQUEST'` in entrambi i casi.

Nulla viene cancellato. Il Report è persistito con il `diffUnified` della Proposal *prima* del tentativo; quando il tentativo fallisce, quello stesso documento passa a `status: FAILED` mantenendo il diff. Poiché la vista di dettaglio sostituisce il corpo con il componente d'errore (Frontend §8.3), il diff resta nei dati ma non nell'interfaccia: l'operazione è fallita e va rilanciata, senza percorsi di successo parziale. Regge su tre argomenti: *coerenza* (ogni altro modo di fallire termina già così), *rarietà* (i permessi insufficienti emergono tipicamente alla configurazione della credenziale, **RF.13**) e *reversibilità* (i dati per un futuro recupero del diff sono già a database).

11.6 Requisiti di qualità sull'esecuzione (RQ.5, RQ.6)

Accuratezza (RQ.5, MPD.6). Nessun componente a runtime può certificare l'accuratezza di un modello: ciò che il backend deve al requisito è non ostacolarne la misura, cioè determinismo in tutto ciò che circonda il modello. Tre proprietà già decise altrove lo garantiscono: instradamento deterministico via `AgentRegistry` (**RV.1**), contesto ancorato a `resolvedSha` (PoC §4.3), prompt come file esterni versionati (**RQ.4**). Rispetto alla PoC, che prepara il *golden set* per il solo Agente Security (§11), l'MVP lo estende ai tre agenti e lo esegue contro il backend dispiegato attraverso il normale `POST /tasks`. La rendicontazione appartiene al Piano di Qualifica; nulla di **RQ.5** entra nel contratto API.

Tempo di risposta (RQ.6, MPD.12). La grandezza misurata è `durationMs`, già presente sull'entità `Report` (fig. 3) ed esposta dall'MVP nell'intestazione (**RF.57**, §12.4): `MPD.12` si legge dai dati di produzione e l'intestazione mostrata all'utente non può divergere

dalla metrica di qualità. La definizione adottata è quella del Piano di Qualifica — “tempo *medio* impiegato da un singolo agente per completare l’analisi ed emettere l’output” — e non quella end-to-end con cui la PoC §11 descrive la propria campagna di misura: le due non coincidono, e la seconda non misura la grandezza che la metrica definisce. Si noti che MPD_12 è definita come una *media* sulle esecuzioni, mentre il tetto di 300 s adottato qui vincola la *singola* esecuzione: è una lettura più stretta della metrica, non una sua contraddizione.

Un’esclusione va dichiarata: per una Task Changelog sospesa su `pendingInput`, il tempo di attesa della risposta dell’utente non rientra nella misura. Altrimenti un utente che impiega dieci minuti a confermare farebbe fallire al sistema la propria metrica di qualità, che misurerebbe l’utente e non il software.

Il tetto di timeout non è un parametro libero. I valori della PoC (90 s per Docs e Changelog, 180 s per Security, fig. 8) sono “un tetto di sicurezza da stringere, non [...] il valore a cui puntare” (PoC §11), e vanno riderivati perché l’MVP ha sette operazioni invece di tre, con `DOCS_README` e `DOCS_API` che leggono molto più repository di `DOCS_INLINE`. Il limite superiore è però più basso di quanto **RQ.6** lascerebbe supporre: la fig. 7 della PoC assegna ad `AgentGatewayService` un **timeout HTTP di 100 s** verso il servizio agenti, sotto sia ai 180 s già configurati per Security sia ai 300 s di **RQ.6**. Finché resta tale, nessun timeout d’agente superiore ha effetto: la conciliazione dei tre numeri è un punto aperto (§11.7).

11.7 Punti aperti verso il servizio agenti

Questa progettazione si ferma dove comincia il servizio agenti. I punti seguenti non sono decisioni che il backend possa prendere da solo e vanno chiusi con chi progetta gli agenti; finché restano aperti, il contratto qui descritto è una proposta sulla sola parte backend.

12 Report: Consultazione, Storico ed Esportazione

Questa sezione progetta **RF.49–RF.64** e **RF.73–RF.74**. Per l’aspetto di **RS.3** relativo alla scrittura della Pull Request si veda ADR-4, §13.

12.1 Riuso dalla PoC

Il Report è già “un envelope unico e generico” (PoC §4.3), e “sia il backend sia il frontend trattano sempre un Report, senza sapere quale agente l’ha prodotto” (PoC §5). L’envelope, i tipi di blocco polimorfi, l’artefatto `Proposal` e le enumerazioni associate sono definiti nella fig. 4 della PoC e **non vengono ridefiniti qui**: le sottosezioni che seguono elencano soltanto ciò che cambia. Manca invece quasi tutto ciò che sta *attorno* all’envelope: l’intestazione non porta nessuno dei dati richiesti da **RF.55–RF.58**, non esiste un titolo, non esiste un’esportazione e non esiste un utente a cui un report appartenga.

12.2 Delta sull'envelope

Nessun campo della fig. 4 viene rimosso o rinominato; tutte le modifiche sono additive tranne dove segnalato.

Perché il contesto viaggia dentro il report. I valori di `context` sono copiati nel documento `Report` all'assemblaggio, non risolti leggendo l'`AnalysisContext`: un report è un registro immutabile di ciò che è stato analizzato e deve restare completo anche se il contesto venisse rimosso — la stessa ragione per cui la PoC ancora il contesto a uno SHA, cioè renderlo interpretabile “anche dopo nuovi commit sul repository” (§4.3). Il Frontend, del resto, non consuma alcun `GET /contexts/:id` (§7.2, Tabella 5) e disegna la riga di metadati come parte della pagina del report (§4.8).

Che cosa vale su un report fallito. Quando la Task fallisce prima che un agente venga invocato — fallimento di orchestrazione o instradamento, Frontend §8.3.1 — `durationMs` e `summary` valgono `null`: meglio di uno zero o di una stringa vuota, che un consumatore potrebbe scambiare per misure reali, e `MPD_12` ignora i valori nulli. Il `title` è invece sempre presente perché deterministico, che è la ragione per cui **RF.51** non è affidato al modello.

12.3 Delta sui blocchi del corpo (RF.59–RF.62)

Categoria e gravità (RF.61). La “categoria” esiste già, sotto un nome diverso per tipo di blocco (`owaspCategory`, `ruleId`, il `kind` stesso): non viene introdotto un campo unico `category`, perché uniformare i nomi costerebbe una modifica al parser di ogni agente senza aggiungere informazione, e il filtro del frontend legge `severity`, l'unico campo che deve essere comune. La gravità è dunque l'unica aggiunta obbligata, su `PolicyViolationBlock`. Per l'avviso di complessità (§10.1) è invece un valore fisso e non un giudizio del modello: il modello decide se l'unità di codice è troppo intricata da documentare, non quanto ciò sia grave.

Rimedio: Snippet o descrizione (RF.62). §10.2 decide per **RF.90–RF.91** che il rimedio è uno Snippet quando il problema è localizzato e una descrizione testuale altrimenti. I due casi devono essere distinguibili nello schema e non concatenati in un unico campo di prosa: da qui `remediationKind`, additivo, che evita di trasformare `remediation` in un tipo unione e di rompere il parser esistente. I numeri di riga si riferiscono al file a `context.resolvedSha`: è l'ancoraggio al commit a renderli ancora significativi quando il report viene riaperto settimane dopo.

Markdown sicuro (RF.59). La conversione in elementi grafici è compito del frontend (§4.8); l'obbligo del backend è consegnare Markdown valido e sicuro da renderizzare. Il contenuto è prodotto da un modello a partire da testo di repository che l'utente non controlla, e Frontend §13.6 avverte che nessun WAF protegge l'applicazione. Il backend normalizza quindi il Markdown *una volta*, all'assemblaggio — blocchi HTML grezzi rimossi, destinazioni `javascript:` e `data:` rifiutate — e persiste il testo normalizzato, così la resa a schermo e l'esportazione PDF consumano la stessa stringa già sicura.

Corpo mai vuoto (RF.60). Un’analisi che non ha trovato nulla produce un report con un `TextBlock` esplicito che lo dice, mai un `body` vuoto: nessun consumatore deve gestire come caso speciale una pagina bianca. La garanzia riguarda i report `COMPLETED`; un report `FAILED` ha per costruzione il corpo vuoto (§11).

12.4 Intestazione del report (RF.55–RF.58)

I campi aggiunti all’envelope in §12.2 servono questi quattro requisiti:

- **Repository (RF.55)** — `context.repoOwner` e `repoName`, presi da `AnalysisContext` (fig. 3), più `repoUrl` perché il frontend renda il nome come collegamento senza ricomporlo.
- **Riferimento e ambito (RF.56)** — `context.branch`, `resolvedSha`, `scopeType` e `paths`. Si mostrano entrambi i riferimenti perché dicono cose diverse: il `branch` è ciò che l’utente ha scelto, lo `SHA` è ciò che è stato letto (la separazione viene da **RF.17**, §6). `paths` è restituito per intero: un’eventuale troncatura è una decisione d’interfaccia, non un limite del contratto.
- **Tempo di esecuzione (RF.57)** — `durationMs`, misurato fra l’invio dell’invocazione all’agente e la ricezione del suo output; esclusi l’attesa in coda, la generazione del PDF e l’attesa della risposta dell’utente. È lo stesso valore su cui si valuta **RQ.6** (§11.6).
- **Data e ora di completamento (RF.58)** — `generatedAt`. Nessun secondo campo `completedAt`: sarebbe lo stesso istante sotto un altro nome. Su un report fallito registra l’istante in cui il fallimento è stato assemblato.

12.5 Collegamento alla Pull Request (RF.63)

Non coperto dalla PoC, che dichiara di non implementare “l’apertura effettiva della Pull Request” (§9.1) e il cui output `Docs` è una `Proposal` la cui “applicazione al repository resta un atto umano” (§6.4). Nell’MVP la `Proposal` guadagna `pullRequestUrl`, scritto dal backend subito dopo aver persistito l’output dell’agente e aperto la PR (**ADR-4**), e validato come URL assoluto verso `https://github.com/` prima della persistenza. Vale `null` solo per le operazioni che non aprono alcuna PR: un report `Docs` la cui apertura sia fallita non raggiunge l’utente con un `null` qui, perché quella `Task` è `FAILED` (§11.5). Coerentemente con **ADR-FE-2** il collegamento è l’elemento primario della vista e l’anteprima del diff resta secondaria: la validazione umana avviene interamente su GitHub, e il sistema non implementa un flusso di approvazione interno (**ADR-AWS-10**).

12.6 Modulo di validazione nel report (RF.64)

Il meccanismo esiste già: è `pendingInput` con `POST /tasks/:id/input` (§9.2.2). **RF.64** non ne introduce un secondo, ma chiede che lo stesso passo in attesa sia visibile *sulla risorsa report*, perché il Frontend elenca un “Modulo di validazione” fra i blocchi di `/reports/:id` (§4.8) oltre all’area interattiva della dashboard (§4.7). Due conseguenze:

1. Il changelog tecnico intermedio di una Task `CHANGELOG_BUSINESS` è persistito come un normale Report con `status: COMPLETED` — è già il modello del Frontend, dove la generazione del changelog tecnico è la fase “il cui esito viene reso disponibile all’utente come report intermedio” (§3.6.1). Porta `operation: CHANGELOG_TECHNICAL`, così titolo e filtro per operazione lo distinguono dal report di business della stessa Task; compare nell’archivio come qualunque altro report. `Task.reportId` è un campo unico e punta al solo report finale: quello intermedio è raggiunto attraverso `pendingInput`. Un caso limite va dichiarato invece che lasciato implicito: se l’utente rifiuta la conferma (**RF.105**), la Task termina `CANCELLED` e `Task.reportId` resta `null` (§11), mentre `pendingInput` torna `null` con la terminazione. Il changelog tecnico già generato resta allora raggiungibile solo dall’archivio, cioè da **RF.49–RF.52**, che sono *opzionali*: se l’archivio non venisse costruito, un report costato un’invocazione dell’agente diventerebbe irraggiungibile. L’alternativa — far puntare `Task.reportId` al report tecnico quando la Task è `CANCELLED` dopo la prima fase — costa una riga di contratto e va decisa esplicitamente.
2. `ReportDto` guadagna `pendingAction`, proiezione in **sola lettura** del `pendingInput` della Task che lo possiede: la decisione dell’utente viaggia sempre su `POST /tasks/:id/input`, così esiste un solo percorso di scrittura qualunque sia la schermata da cui agisce.

Punto da allineare con il Frontend. §9.2.2 chiama le varianti `WORK_ITEM_QUALITY` e `TECHNICAL_CHANGELOG_CONFIRMATION`, con quest’ultima che porta il testo dell’anteprima; il Frontend (§7.4) le chiama `INCOMPLETE_TASKS` e `BUSINESS_CONFIRMATION`, e la seconda porta `technicalReportId`. Le due forme non sono compatibili e vanno riconciliate in un’unica decisione.

Sulla conferma la posizione di questo documento è che valga la forma del Frontend, e l’argomento è testuale: **RF.103** chiede un comando “all’interno del report del *Change-log* tecnico”. Se il comando sta *nel* report, il report tecnico deve esistere come risorsa indirizzabile, e ciò che viaggia nel payload è il suo identificativo; un’anteprima in linea duplicherebbe un contenuto già persistito e raggiungibile. Sul nome dell’altra variante non esiste un argomento altrettanto stringente — `WORK_ITEM_QUALITY` evita di legarsi alla terminologia di un provider e di confondersi con l’entità `Task` del backend — e la scelta resta aperta.

12.7 Storico, consultazione e controllo di accesso (RF.49–RF.53)

`GET /reports (§??)` è già previsto con i filtri `operation`, `from` e `to` (PoC §7.2, mappato su **RF.49–RF.51**), ma la PoC non ne progetta la forma di risposta. La forma di sintesi è invece già definita dal Frontend (§7.4, `ReportSummaryDto {id, operation, status, generatedAt, title}`) e viene adottata invece che reinventata: `body`, `proposal` ed `error` sono esclusi dalla proiezione, perché un archivio di N report non deve trasportare N envelope completi. L’elenco è ordinato per `generatedAt` discendente.

Titolo descrittivo (RF.51). `title` è composto dal backend all’assemblaggio nella forma *nome leggibile dell’operazione* — `owner/repository@branch` (per esempio “Scansione OWASP — SkynetUniGroup/Code_Guardian@main”). Non è generato dal modello: un

titolo prodotto dall'LLM sarebbe non deterministico, richiederebbe un proprio ramo di validazione e mancherebbe proprio sui report **FAILED**. Il nome leggibile dell'operazione viene dallo stesso catalogo che alimenta `GET /operations` (**RF.32–RF.34**). Due esecuzioni della stessa operazione sullo stesso riferimento condividono il titolo e si distinguono per la data: costo accettato, e l'unico caso davvero ambiguo — i due report di una Task `CHANGELOG_BUSINESS` — è risolto dall'*operation* distinta (§12.6).

Proprietà e controllo di accesso (RF.53). La PoC non può coprirlo: il suo login “firma sempre lo stesso JWT per un utente fisso” (§7.2), quindi non esiste un'identità con cui confrontare qualcosa. Nell'MVP entrambi gli endpoint sono protetti da JWT e il guard confronta `Report.userId` con il claim `sub`; in caso di mancata corrispondenza la risposta è **404**, non **403**, perché un **403** confermerebbe l'esistenza del report di un altro utente. La stessa regola vale per l'esportazione (§12.8): un confine di autorizzazione che vale su un endpoint e non sul suo gemello non è un confine. Il `userId` è denormalizzato su `Report` e non risolto attraverso `Task`, così la query dell'archivio resta una singola lettura indicizzata su (`userId`, `generatedAt`).

Un solo endpoint basta per entrambe le vie d'accesso: la selezione avviene dall'archivio (**RF.49**) o dal comando “Visualizza report” della dashboard (Frontend §4.7), e la pagina di dettaglio è “un unico componente condiviso” raggiungibile da entrambe (Frontend §3.7).

12.8 Esportazione in PDF (RF.73–RF.74)

Della PoC si eredita il solo percorso: `GET /reports/:id/export?format=md` compare una volta nel Listing 1 di §4.2 e mai altrove — né fra gli endpoint pubblici (§7.2, Tabella 4) né nella sintesi del perimetro (§2.2, Tabella 1) — e nulla di ciò che gli sta dietro è progettato. Il formato Markdown decade: l'unico valore ammesso è `pdf`, ciò che **RF.73** richiede e l'unico che il Frontend consuma (§7.2: “l'endpoint di esportazione report utilizza esclusivamente il formato PDF”, con il contratto fissato in Tabella 5 come streaming binario). Due vincoli sono adottati e non ridecisi: Frontend §13.5 vieta gli URL pre-firmati e impone lo streaming *same-origin* attraverso il backend; la Progettazione Infrastruttura AWS (Tabella 7) vieta il rendering via browser headless e impone una libreria di composizione programmatica (`pdfkit/pdf-lib`).

Generazione (RF.73). Endpoint protetto da JWT, stessa regola di proprietà di **RF.53**. Il PDF è composto programmaticamente dal `ReportDto` persistito — mai dal DOM renderizzato — così esportazione e schermata leggono la stessa fonte di verità; è archiviato nel bucket S3 privato (AWS §7.1, Tabella 15) con chiave derivata dall'identificativo del report e restituito con `Content-Type: application/pdf` e `Content-Disposition: attachment`. È generato alla prima richiesta e poi riusato, dato che un report è immutabile; poiché quel bucket elimina gli oggetti dopo 30 giorni, un'assenza in cache si limita a rigenerare, il che la rende un'ottimizzazione e mai una dipendenza. `format` è validato contro l'unico valore ammesso e respinto con **400**; l'esportazione di un report **FAILED** è respinta con **409**, perché il suo corpo è vuoto per costruzione.

Fallimento (RF.74). Frontend §8.4 fissa un comportamento che diverge da ogni altro fallimento del sistema: errore transitorio nella pagina, report che **resta COMPLETED**, blocchi già renderizzati non sostituiti. L'esportazione è quindi di sola lettura e non tocca alcuno stato: nulla viene scritto su S3, l'endpoint risponde 500 con `{ code: 'EXPORT_FAILED', message }` e nessun evento WebSocket viene emesso. Il valore non entra nell'enumerazione di §11.2 perché non appartiene alla famiglia di UC27.

Poiché la risposta deve poter fallire *dopo* il momento in cui i primi byte verrebbero inviati, il PDF è composto interamente in memoria e solo allora scritto nel corpo: è ciò che garantisce che il browser non riceva mai un file troncato che sembra uno scaricamento riuscito. La scelta pesa però dove la Progettazione AWS registra già un rischio accettato e non verificato (Tabella 20: task da 0,5 vCPU e 1 GB esteso alla generazione dei PDF “senza revisione dedicata”), da confermare con un test di carico su esportazioni concorrenti prima del rilascio.

13 Decisioni Architettureali

ADR-1 – ADR-3: Riuse identiche dalla PoC (§10): il token GitHub non lascia mai il backend (ADR-1); l'LLM è raggiunto dietro l'interfaccia `LLMProvider`, con `BedrockProvider` pronto ma non attivo (ADR-2); l'esecuzione degli agenti passa sempre da una coda, mai dal ciclo della richiesta HTTP (ADR-3).

ADR-4: La scrittura su GitHub resta un'operazione esclusiva del backend NestJS. L'agente Python continua a produrre solo una Proposal (diff) attraverso la facade read-only invariata (§6, ADR-1). Una volta ricevuto e persistito quell'output nel Report, è il backend a invocare l'endpoint di scrittura delle API GitHub (branch + commit + apertura PR), usando un token con scope repo dedicato. L'apertura è automatica e immediata al termine dell'esecuzione dell'agente: Code Guardian non implementa un passaggio di conferma umana preliminare. La validazione della modifica proposta avviene interamente su GitHub, tramite il normale flusso di review/merge della PR (RF.63).

Questa decisione recepisce ADR-AWS-10 (Progettazione Infrastruttura AWS, §13.1) e ADR-FE-2 (Progettazione Frontend, §11.3).

Perché RS.3 resta rispettato. Su due livelli indipendenti: applicativo (il token di scrittura non lascia mai NestJS, stesso principio di ADR-1) e di rete (il container agents non ha alcuna rotta verso l'esterno — zero egress — quindi non potrebbe raggiungere GitHub nemmeno se il token gli fosse passato per errore).

ADR-5: L'MVP promuove a requisito obbligatorio un'autenticazione reale (**RF.1–RF.9, RS.1**), assente nella PoC. Decisione: email e password, con hashing Argon2id e sessione stateless tramite JWT (HS256, scadenza 8 ore), senza refresh token; il ruolo dell'utente viaggia come claim nel token, evitando una query al database su ogni richiesta autenticata (dettagli in §3).

Perché Argon2id e non bcrypt. È l'algoritmo di hashing per password raccomandato più di recente, con resistenza nativa ad hardware specializzato (GPU/ASIC) che bcrypt non offre nella stessa misura.

Perché nessun refresh token. Un refresh token serve a rinnovare silenziosamente una sessione a vita breve senza richiedere un nuovo login. Qui non esiste uno scenario in cui serva: la Progettazione Frontend scarta già l'intera sessione a ogni ricaricamento della pagina (ADR-FE-6), quindi un token che sopravvive più a lungo del browser aperto non offrirebbe alcun beneficio percepibile, a fronte della complessità aggiuntiva di rotazione, storage e revoca.

Da non fare: introdurre un token store lato server (sessioni tradizionali). Aggiungerebbe stato da sincronizzare in un'infrastruttura già vincolata a un'unica istanza del backend (**Desired Count:** 1, Progettazione Infrastruttura AWS), senza che alcun requisito lo richieda.

ADR-6 — Un solo token per codice e Task Management: Un solo servizio esterno configurabile nell'MVP, GitHub, e un solo token per credenziale, che copre sia l'accesso al codice (**RF.11**) sia l'accesso al sistema di Task Management (**RF.12**). Non sono due integrazioni: sono la stessa API, letta per due scopi diversi attraverso gli stessi tre tool della facade (`read_tree`, `read_file`, `read_issues`, §5).

La decisione discende da una scelta di team già presa a monte e non riaperta qui: GitHub Issues coincide con il sistema di Task Management del progetto (§1.2). Fissata quella premessa, la conseguenza sul lato credenziali è quasi automatica — lo scope `repo` di un Personal Access Token GitHub copre già, per definizione del provider, sia il contenuto del repository sia le sue Issue, quindi non esiste una configurazione in cui le due funzionalità richiederebbero permessi realmente separati.

ADR-7 — Separazione tra chiave di cifratura e credenziale cifrata: Ogni `ServiceCredential` è cifrata con una chiave dedicata al record, derivata tramite HKDF da `CREDENTIAL_MASTER_KEY` e da un `salt` generato casualmente per quella credenziale — non con la chiave master applicata direttamente (§4.3). La chiave master vive in AWS Secrets Manager, cifrata a riposo dalla CMK KMS condivisa: un sistema diverso da MongoDB Atlas, dove risiede il ciphertext.

Perché una chiave derivata per record. Cifrare ogni credenziale direttamente con la chiave master renderebbe una singola chiave compromessa — o un bug che la esponga nei log — sufficiente a decifrare ogni credenziale di ogni utente. Una chiave derivata per record contiene il danno: comprometterne una espone solo quella credenziale, mai la master key né le altre.

Perché HKDF e non Argon2id/scrypt (**RS.1**). Quelle funzioni sono deliberatamente lente, pensate per derivare chiavi da segreti a bassa entropia come una password umana. La chiave master è già ad alta entropia, generata da AWS Secrets Manager: una funzione di derivazione lenta non aggiungerebbe sicurezza, solo latenza a ogni lettura di credenziale.

Punto	Che cosa va deciso
Valori di <code>ErrorKind</code>	I cinque valori nuovi rilevati dall'agente (<code>CONTEXT_TOO_LARGE</code> , <code>CONTEXT_RESOURCE_MISSING</code> , <code>CONTEXT_RESOURCE_INVALID</code> , <code>LLM_RATE_LIMITED</code> , <code>READABILITY_TOO_LOW</code>) sono un'estensione dell'output dell'agente, non del backend; per gli ultimi due restano inoltre da definire messaggio e azione secondaria in coordinamento con chi cura la Progettazione Frontend (Tabella 7, che non ha ancora una voce corrispondente per nessuno dei due).
Conciliazione dei timeout	100 s (<code>AgentGatewayService</code>), 90/180 s (per agente), 300 s (RQ.6). Chi possiede il valore, e chi lo passa a chi nell'invocazione.
Sospensione e ripresa	<code>invokeAgent</code> è una chiamata HTTP sincrona: una sospensione in attesa dell'utente (§9.2.2) dura minuti e non può viverci dentro. Il contratto di sospensione/ripresa fra i due servizi non è definito da nessuna parte.
Avviso di complessità	RF.61 richiede categoria e gravità anche per l'Agente Docs, che nella PoC "emette un avviso strutturato" (§9.1) senza che se ne definisca la forma. Il blocco proposto in §12.3 va confermato da chi scrive il parser.
Gravità sulle violazioni di policy	<code>PolicyViolationBlock</code> non porta oggi alcuna <code>Severity</code> , che RF.61 invece richiede. Le due figure della PoC non concordano nemmeno su <code>remediation</code> : presente in fig. 3, assente in fig. 4.
Chi valorizza <code>durationMs</code>	<code>AgentState</code> porta <code>tokens_consumed</code> (fig. 10), quindi <code>Report.tokensConsumed</code> nasce dall'agente; per <code>durationMs</code> nessun documento dice chi lo scrive. Questa progettazione assume il backend, perché è il backend a persistere il Report (fig. 9), ma va confermato.

Tabella 6: Punti da chiudere con il servizio agenti prima della codifica.

Campo	Stato	Nota
<code>title: string</code>	nuovo	RF.51. Composto dal backend, mai dal modello.
<code>context: {...}</code>	nuovo	RF.55–RF.56. Oggetto denormalizzato, campi in §12.4.
<code>durationMs</code>	esposto	Esiste già sull'entità <code>Report</code> (fig. 3) ma non nel <code>ReportDto</code> (PoC §7.4). L'M-VP lo espone e ne fissa la semantica (RF.57); vale <code>null</code> se nessun agente è stato invocato.
<code>summary</code>	modificato	Da <code>string</code> a <code>string null</code> : su un report <code>FAILED</code> nessun output del modello esiste.
<code>generatedAt</code>	invariato	RF.52, RF.58. Se ne fissa solo la serializzazione: persistito in UTC, esposto in ISO 8601 con offset esplicito, scritto una volta sola all'assemblaggio.
<code>proposal.pullRequestUrl</code>	nuovo	RF.63. <code>string null</code> .
<code>pendingAction</code>	nuovo	RF.64. Proiezione in sola lettura di <code>pendingInput</code> (§12.6).
<code>error.kind</code>	dominio esteso	§11.2. Il campo e il suo nome restano quelli della PoC.
<code>body: Block[]</code>	invariato	L'array è restituito già ordinato per <code>Block.order</code> (fig. 4); non esiste paginazione interna al corpo.
<code>userId</code>	nuovo	Solo in persistenza, non nel DTO: serve al controllo di proprietà e all'indice (<code>userId</code> , <code>generatedAt</code>) (§12.7).
<code>tokensConsumed</code>	invariato	Persistito e non esposto, come nella PoC.

Tabella 7: Modifiche all'envelope `Report` rispetto alla fig. 4 della PoC.

Tipo	Stato	Nota
TextBlock	invariato	RF.59. Cambia solo un obbligo del backend, non il tipo: il Markdown è normalizzato all'assemblaggio.
FindingBlock	due campi nuovi	RF.62. <code>remediationKind: 'SNIPPET' 'TEXT'</code> e <code>remediationLanguage?: string;</code> <code>remediation: string</code> resta invariato e continua a portare il testo o il codice.
PolicyViolationBlock	un campo nuovo	RF.61. <code>severity: Severity</code> : oggi il blocco non ne porta alcuna, mentre il requisito la richiede per ogni elemento strutturato. Va chiarito contestualmente che ne è di <code>remediation</code> , che la fig. 3 dichiara e la fig. 4 no.
ComplexityWarningBlock	nuovo tipo	RF.61 per l'Agente Docs. <code>kind, filePath, startLine, endLine, explanation, severity</code> fisso a <code>info</code> . Stessi nomi di campo del <code>FindingBlock</code> .
Severity	invariato	<code>info low medium high critical</code> . L'MVP non ne cambia i valori: <code>info</code> è il livello che il Frontend riserva proprio agli avvisi di complessità dell'Agente Docs (§5.2, Tabella 4).
ChangelogItemBlock	invariato	—

Tabella 8: Modifiche ai blocchi del corpo rispetto alla fig. 3 della PoC.